ASC 2025 Proposal of XTU

Team members: Jiaru Xia, Zhixiong Zhang, Peng Qiu, Liangshang Wu, Fuxing Gong
Advisor: Peiying Lin

Contents

1. Supercomputing Activities 3
1.1. The HPC Platform of XTU 3
1.2. HPC Center Activities 4
1.3. Campus Activities 4
1.4. Research and Applications 4
1.5. Key Achievements 5
2. Our Big Family 7
2.1. Setup of the Team 7
2.2. Introduction of Members 7
2.3. Team Slogan 9
3. Design of Our HPC System 10
3.1. Cluster Overview 10
3.2. Hardware Configurations 10
3.3. Software Configurations 11
3.4. Performance Analysis 11
3.5. Power Specifications 11
4. HPL Test and Optimization 13
4.1. Analysis of HPL 13
4.2. Performance Optimation Methods 13
4.3. Machine Performance Analysis 17
4.4. Experiment Results 17
4.5. Summary 18
5. HPCG Test and Optimization 20
5.1. Analysis of HPCG 20
5.2. Performance Optimation Methods 20
5.3. Experimental Results 25
5.4. Summary 28
6. Optimization for AlphaFold3 Inference 29
6.1. Introduction 29
6.2. Related work 29
6.3. GPU Optimization Methods 31

6.4. Implementation and Optimization of CPU 34
6.5. Experiments 45
6.6. Optimization Plans 48
6.7. Conclusion 51
7. RNA m^5C Modification Site Detection and Performance Optimization 52
7.1. Introduction 52
7.2. Related Work 52
7.3. Proposed Methods 53
7.4. Experiments 60
7.5. Optimization Plans 62
7.6. Conclusion 64
Acknowledge 65
References 66

1. Supercomputing Activities

Xiangtan University(XTU) is a prestigious comprehensive national key university and a "Double First-Class" construction university of the country. Standing majestically in the hometown of the great leader Mao Zedong, XTU is a place of great renown. XTU not only has modern teaching facilities, but also is equipped with an advanced high-performance computing center. In terms of high-performance talent cultivation, the school provides students with abundant high-performance learning resources, including cutting-edge professional courses, high-level academic lectures, innovative research projects, and a powerful high-performance computing platforms. These resources not only broaden students' horizons, but also provide them solid academic and technical support. After years of meticulous cultivation, the student teams of XTU participate in the ASC competition for the first time this year. Next, we will introduce the university's supercomputing platform and supercomputing-related activities.

1.1. **The HPC Platform of XTU.**

Xiangtan University's Super Heterogeneous Parallel Computing Cluster is a key project supported by the provincial finance under the "14th Five-Year Plan" for new infrastructure construction. The cluster includes ClusterEngine high-performance computing platform and AIstation artificial intelligence platform, and is open to all teachers and students of the university. The computing capacity of the cluster is the theoretical computing capacity of 509 trillion times of CPU double-precision peak performance (6,112 physical cores) and 1552 trillion times of GPU single-precision theoretical computing capacity. The overall logical framework design diagram of the cluster is shown in the Figure 1.

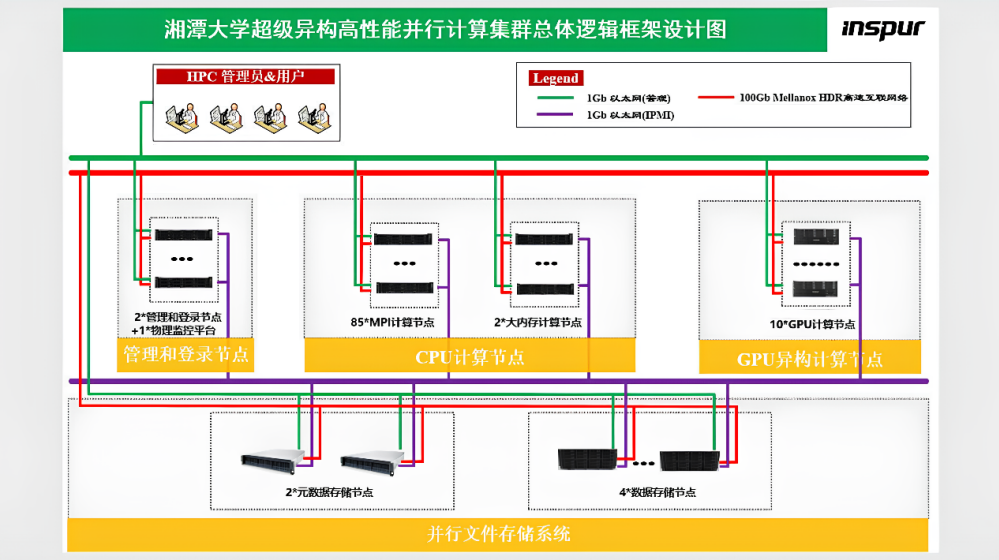


Figure 1. *The structure and network of HPC platform at XTU*

The ClusterEngine high-performance computing platform consists of 1 management node, 1 login node, 1 physical monitoring platform, 85 general-purpose computing nodes, 2 large-memory

computing nodes, totaling 5472 CPU cores and a total memory capacity of 44.5TB. The nodes are interconnected through Gigabit Ethernet and Infiniband. The management and login nodes are each equipped with 16 CPU cores and 64GB of memory.

The cluster has two resource partitions: CU and FAT partitions. The CU partition is configured with 80 general-purpose computing nodes, each with 64 Intel 8358 CPU cores and 512GB of memory, totaling 5,120 CPU cores. The FAT partition is configured with 2 large-memory computing nodes, each with 16 Intel 6334 CPU cores and 1TB of memory.

The main function of the platform is to provide high-performance scientific and engineering computing services to the entire university, meeting the needs of various disciplines for large-scale data processing and large-scale scientific computing. The platform's main application areas include providing high-performance computing environments for mathematics, statistics, mechanics, physics, chemistry, chemical engineering, biology, earth science, aerospace science and technology, computer science, and many other disciplines. The platform supports a wide range of application software, such as MATLAB, OpenFOAM, ABINIT, GROMACS, NAMD and so on.

The AIstation artificial intelligence platform consists of 10 GPU computing nodes, with a total of 640 CPU cores, 80 NVIDIA A100 acceleration cards, a total memory capacity of 5TB, and shared with the parallel storage and high-performance computing platform. The total capacity is 1.3PB. The GPU's measured peak speed can reach 888.9 trillion operations per second. This platform supports single-machine and distributed running of mainstream AI frameworks as well as task visualization.

1.2. **HPC Center Activities.**

Xiangtan University actively conducts academic activities and research projects related to supercomputing. And, the university has participated in the construction of the Hunan Computing Power Network Integration Collaborative Innovation Platform, promoting the interconnection and networking of computing infrastructure and the efficient utilization of computing resources. Through these supercomputing platforms and related activities, Xiangtan University not only provides students with abundant learning resources but also helps the university to carry out scientific research innovation and talent cultivation in the field of high-performance computing.

1.3. **Campus Activities.**

To cultivate talents in the field of high-performance computing, the school provides various activities. In terms of courses, the relevant courses offered by the school include *Parallel Programming, Computer Organization Principles, Syntax and Semantics of Compilers, Computer System Architecture, Operating Systems, Principles of Computer Networks, Linux System Analysis and Programming, Algorithm Design and Analysis, Mathematical Analysis, C Programming, Graph Theory and Combinatorial Mathematics, Data Structures, Artificial Intelligence*, etc., which are a wealth of courses. In terms of regular training, through platforms such as EasyHPC, the school invites previous ASC participants to provide regular practice and training for students. Separate classrooms and servers are also provided for students. In terms of interest groups, in addition to forming ASC competition teams, the school's ACM team also actively participates.

1.4. **Research and Applications.**

The research and application achievements of XTU in the field of supercomputing are remarkable, including parallel solvers, the development of high performance computing software, the optimization of computer system architecture, the design of high performance computing algorithms and the application of high performance computing in scientific research, engineering design, industrial production and other fields. The following is only a brief list.

For example, the JXPAMG solver jointly developed with the Beijing Institute of Applied Physics and Computational Mathematics has become the default linear solver of JASMIN, a high-performance parallel computing software platform, with its algorithm innovation, efficient parallel performance and multi-field applicability, and its comprehensive performance is better than the international popular similar solvers [1]. The solver has greatly improved the computational efficiency of some defense application model problems, and has an important influence in the field of scientific calculation and engineering simulation.

The chemical flooding model is widely used in the numerical simulation of complex reservoir because of its high oil recovery [2]. Therefore, for the numerical simulation of chemical flooding reservoir, CPU-GPU heterogeneous parallel discretization technology is used to discretized the chemical flooding model. The numerical experiment results show that the CPU-GPU heterogeneous parallel discretization method has a good acceleration effect. For example, for the calculation example with a grid size of about 1.25 million, The speed of the new algorithm is 3.99 times faster than that of the serial algorithm. Then, a non-smooth aggregation algebraic multi-grid preconditioned conjugate gradient iterative algorithm (UA-AMG-CG) based on CPU-GPU heterogeneous system with low operator complexity, high acceleration ratio and good robustness is designed for water phase pressure equation and component concentration equation, and the corresponding program module is developed.

1.5. **Key Achievements.**

As mentioned above, XTU's research in supercomputing includes many aspects, and have published multiple papers in international journals and conferences in the fields of mathematics and computer science and so on. Such as *CCF Transactions on High Performance Computing, Applicable Analysis, Mathematical Methods in the Applied Sciences, International Conference on Management of Data(SIGMOD) International Conference on Very Large Data Bases(VLDB), IEEE Transactions on Parallel and Distributed Systems(TPDS)* and other journal conferences, here is a brief overview of two of the outcomes:

Innovation 1: Parallel Reservoir Simulators on Tianhe-2 [3]. Parallel reservoir simulators are now widely used with availability of super computers. Modern massively parallel supercomputers demonstrate great power for simulating large-scale reservoir models. However, improving scalability and efficiency for fully implicit methods on emerging parallel architectures is still challenging. In this work, we present a robust discretization together with a parallel linear solver algorithm; and we explore the parallel implementation on the world's fastest supercomputer Tianhe-2. Starting with a general compositional model, we focus on the black oil model and developed Parallel eXtension Framework for parallelizing the serial simulator. A parallel preconditioner based on fast auxiliary space preconditioning (FASP) is applied to solve the Jacobian system arising from the fully implicit discretization. The parallel simulator was validated using large-scale black oil benchmark problems, for which parallel scalabilities were tested. Giant reservoir models with over 100 million grid blocks have been simulated within a few minutes, and test the strong scalability of AMG solver with 1 billion unknown. We also demonstrate the parallelization and acceleration using Intel Xeon Phi coprocessors. In the end, the efficiency of the parallel simulator is illustrated by a giant reservoir using up to 10,000 cores, for which the CPU and communication time are summarized for the linear and nonlinear algorithms.

Innovation 2: Efficient Algorithms for the Maxwell Eigenvalue Problem [4]. The Maxwell eigenvalue problem is a basic and important computational model in computational electromagnetism, e.g., in regard to electromagnetic waveguides and resonances in cavities. This work proposed two two-grid methods for the Maxwell eigenvalue problem. The new two-grid schemes reduce the solution of the Maxwell eigenvalue problem on a fine grid to one linear indefinite Maxwell equation on the same fine grid and an original eigenvalue problem on a much coarser grid. The new schemes, therefore, save total computational cost. The error estimates reveal that the two-grid methods maintain asymptotically optimal accuracy, and the numerical experiments presented confirm the theoretical results.

2. Our Big Family

Under the guidance of our advisor and after undergoing relevant training and selection, we established the ASC competition team. As the first ASC competition team from our school, we engaged in extensive preparatory work over an extended period and received substantial assistance from both campus and external communities. We are grateful for their technical support and valuable advice, which enabled us to participate in the competition smoothly. The following sections detail the team composition and current status.

2.1. **Setup of the Team.**

This competition was initiated by Professor Peiying Lin, who teaches *Parallel Programming* and *Computer Organization Principles*. Professor Lin earned her Ph.D. from Hunan University and specializes in high-performance computing research. Starting in September 2024, she conducted promotional activities through courses and lectures at the university while designing selection assessments. In December 2024, after candidates passed these assessments, our supercomputing competition team was officially formed. Following the team's establishment, Professor Lin organized specialized training sessions covering foundational concepts, parallel computing techniques, and optimization strategies. Additionally, she repeatedly invited former ASC competition participants from other institutions to provide experiential guidance to our team as shown in Figure 2.

Figure 2. *ASC training*

2.2. **Introduction of Members.**

Our competition team consists of five official members and two reserve members, under the guidance of advisor Peiying Lin. All participating students are undergraduates majoring in Computer Science and Technology, namely Jiaru Xia, Zhixiong Zhang, Liangshan Wu, Peng Qiu, and

Fuxing Gong. And, the photo of our team is shown in Figure 3. Below is a detailed introduction of each team member.

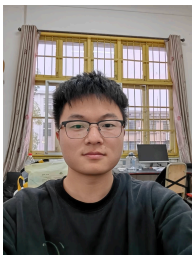
Figure 3. *Team members*

Jiaru Xia, team leader, possesses exceptional organizational and coordination skills, solid computer science foundations, and a strong spirit of challenge. She demonstrates proficiency in Linux systems and Git version control tools, with expertise in MPI and OpenMP multi-thread programming as well as CUDA GPU acceleration technologies. Her notable achievements include winning the National Second Prize in the China Collegiate Computing Competition and receiving a Merit Award in the National College Student Information Security Contest. In the ASC competition, as team leader, she not only oversees overall project management but also spearheads three critical tasks: (1) Architecture design and performance optimization of HPC systems to ensure computational efficiency and system stability; (2) Optimization of HPCG benchmarking to enhance computational performance; (3) Participation in performance optimization for RNA m^5C modification site detection.

Zhixiong Zhang, a passionate full-stack computer enthusiast, demonstrates expertise in Linux and Git with strong capabilities in parallel computing and HPC. Possessing foundational knowledge in artificial intelligence, he has conducted in-depth research in deep learning and developed a profound understanding of hardware architectures. His development experience includes creating a graduate employment prediction system and a quantitative financial advisory system. In this competition, he leads the CPU deployment and optimization design implementation for AlphaFold3. Additionally, he contributes to performance optimization analysis for HPL and HPCG, while assisting in GPU deployment and optimization

efforts for AlphaFold3.

Liangshan Wu, a passionate enthusiast of mathematics and algorithms with rigorous logical thinking, is proficient in Linux and Git. He has mastered the C++ and Python programming languages and possesses a solid foundation in algorithms, excelling in performance optimization and data structure design. Additionally, he has acquired expertise in machine learning and deep learning applications. During this competition, his primary responsibility focused on performance optimization for RNA m^5C modification site detection, where he significantly improved algorithm processing speed and accuracy through parallel computing techniques.



Peng Qiu possesses a solid foundation in computer science and demonstrates a strong spirit of exploration. He has proficient command of Linux and Git systems, excels in C/C++ programming, and is well-versed in parallel programming with MPI and OpenMP. Additionally, he maintains an in-depth understanding of AI technologies and their practical applications. Among his notable achievements is the development of a supermarket shopping system. In this competition, his primary responsibilities included GPU deployment implementation of AlphaFold3 and the design/execution of optimization solutions.

Fuxing Gong, passionate about computer science with strong learning capabilities. Proficient in Linux and Git environments, skilled in C/C++, and Java programming languages. Demonstrates exceptional algorithm design and implementation abilities, quickly adapting to diverse programming environments. Possesses hands-on experience in large-scale parallel computing and multithreaded programming. Previously developed simple game prototypes. In this competition, he primarily led the design and execution of HPL benchmarking while contributing to GPU-based performance optimization for the AlphaFold3 model.

2.3. **Team Slogan.**

Unlimited Algorithms, Fearless Supercomputing, Victory Walks with Us.

3. Design of Our HPC System

This section introduces the design of the HPC system. It begins with a description of the HPC system architecture, followed by detailed discussions of hardware configuration, software configuration, and energy consumption, concluding with an comprehensive performance analysis.

3.1. **Cluster Overview.**

The theoretical design of the HPC cluster is illustrated in Figure 4, which features a five-node computing architecture, satisfying the minimum requirement of three computational nodes. Each node is equipped with a dual-processor server configuration augmented by two GPU accelerators, interconnected through Gigabit Ethernet and high-speed InfiniBand network solutions to ensure optimal data throughput and low-latency communication between nodes.

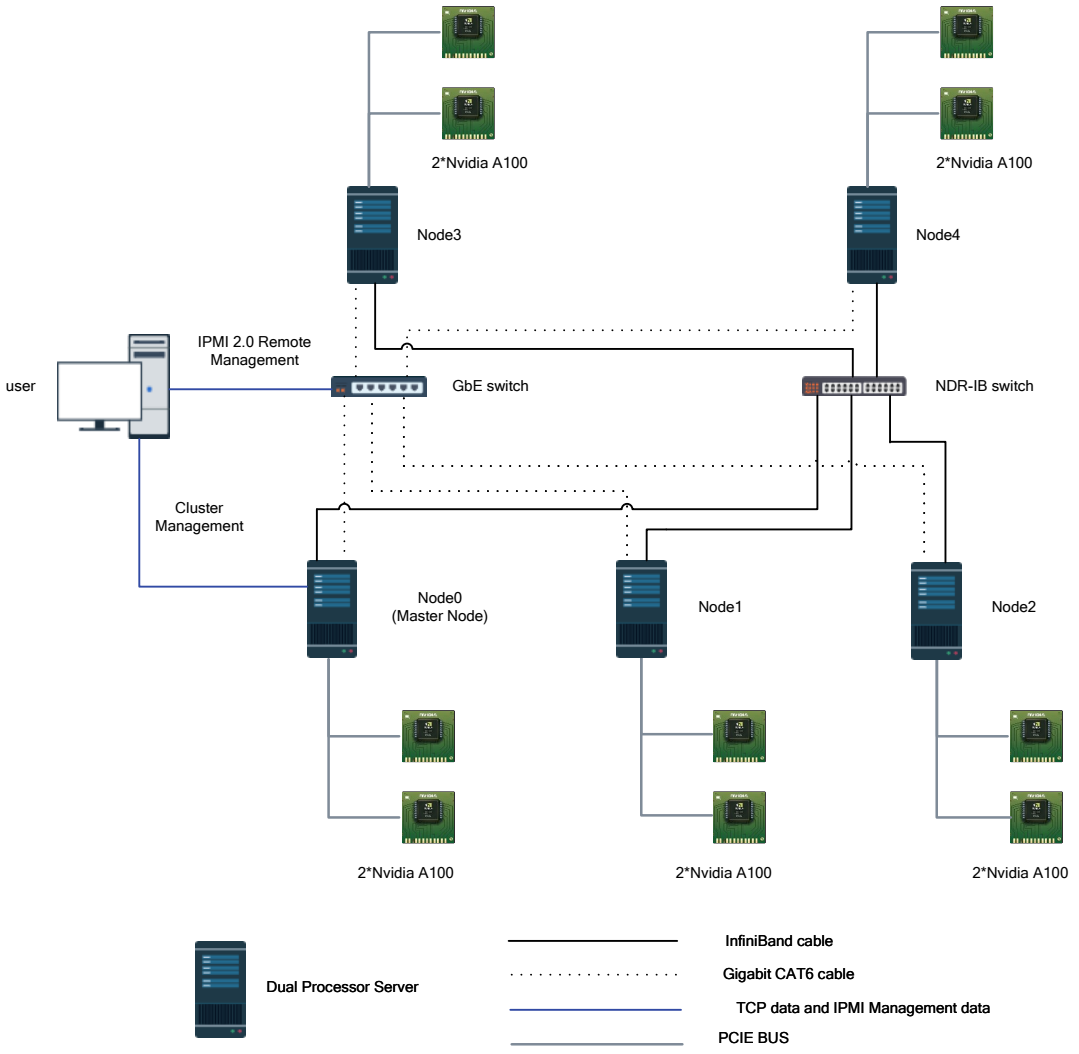


Figure 4. *Cluster Diagram*

3.2. **Hardware Configurations.**

This section outlines the hardware configuration of the HPC system, including the type of compute nodes, processors, memory, storage, and network infrastructure. The complete technical specifications are systematically presented in Table 1.

Table 1. *Hardware resources*

Item	Name	Quantity
Server	Dual Processor Server	5
GPU	Nvidia A100 PCI-E	10
HCA card	NDR200	5
Switch	GbE switch	1
	NDR-IB switch	1
Cable	Gigabit CAT6 cables	6
	InfiniBand cable	5

3.3. **Software Configurations.**

This section covers the software configuration of the HPC system, as shown in Table 2, including the operating system, middleware, and computational libraries that support parallel processing and resource management. These integrated systems synergistically enable optimized parallel processing and intelligent resource allocation across distributed computing clusters.

Table 2. *Software resources*

Item	Name	Remarks
Operating system	Ubuntu 22.04.4 LTS	
Compiler	GNU Compiler	Version 11.4.0
Parallel Development Environment	Intel Parallel Studio XE	Version 19.0
	PGI	Version 18.10
	CUDA Toolkit	Version 12.4
MPI	openmpi	Version 4.1.2
	intel_mpi	Version 2025.0.4
Mathematical Library	Openblas, CuBLAS, AOCL-BLAS, MKL	
Application software	Anaconda	Version 2.3

3.4. **Performance Analysis.**

(1) The overall CPU theoretical peak performance is:

$$1.9\,\text{GHz} \times 144\,(\text{cores/CPU}) \times 2\,(\text{CPUs/node}) \times 5\,(\text{nodes}) \times 32\,(\text{floating point execution unit}) = 87552\,\text{Gflops} = 87.552\,\text{Tflops}$$

(2) The overall GPU theoretical peak performance is:

$$7\,\text{Tflops} \times 10 = 70\,\text{Tflops}$$

3.5. **Power Specifications.**

Our HPC cluster comprises five computing nodes. This section provides a detailed analysis of power consumption patterns for both GPU and CPU operations at individual node level and cluster-wide scale, as detailed in Table 3. As can be seen from the table, for a single node, **the**

CPU computing power consumption is $614W+20W=634W$**, and the GPU computing power consumption is** $150W+2\times250W=650W$**, both of which meet the power consumption requirement of less than** $2000W$**. For the entire cluster, the CPU computing power consumption is** $3850W$**, and the GPU computing power consumption is** $3830W$**, which also meet the power consumption requirement of less than** $4000W$**.**

Table 3. *Power description*

Mode	Name		Power consuption
CPU Computing	Dual Processor Server	CPU: Intel® Xeon® 6766P Processor * 2(250w * 2) Memory: 32GB * 16, DDR5, 6400 MT/s(5w * 16) Hard disk: 480GB SSD SATA * 1(4w * 1) Motherboard Power Consumption(30w)	614w * 5
		NVIDIA A100 PCI-E	20w * 10
		NDR200	30w * 5
		GbE switch	30w
		NDR-IB switch	400w
		Total	3850w
GPU Computing	Dual Processor Server	CPU: Intel® Xeon® 6760P Processor * 2 Memory: 32GB * 16, DDR5, 6400 MT/s Hard disk: 480GB SSD SATA * 1 Motherboard Power Consumption	150w * 5
		NVIDIA A100 PCI-E	250w * 10
		NDR200	30w * 5
		GbE switch	30w
		NDR-IB switch	400w
		Total	3830w

4. HPL Test and Optimization

This section is about the testing and analysis of HPL. All the test results are based on the hardware platform as shown in Table.7.

4.1. **Analysis of HPL.**

The High-Performance Linpack (HPL) Benchmark serves as a standardized computational tool for evaluating the floating-point performance capabilities of high-performance computing (HPC) systems. By solving large-scale systems of linear equations **Ax**=**b**, i.e., Eq.4.1, where matrix **A** represents an $n \times n$ large-scale dense matrix, this benchmark quantifies system performance in floating-point operations per second (FLOPS). This method is one of the internationally recognized standards for assessing supercomputer performance.

One way to solve Eq.4.1 is Gaussian elimination, which first results in a triangular matrix by first converting the **A** in the **Ax**=**b** equation into a triangular matrix by elimination.

Suppose the solution problem is:

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \cdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \cdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \dots \\ b_n \end{bmatrix} \tag{4.1}$$

The basic idea of solving the equation about **x** is as follows.

First, the matrix transformation of **A** is carried out to obtain the following form:

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ 0 & a_{22} & \cdots & a_{2n} \\ \dots & \dots & \dots & \cdots \\ 0 & 0 & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \cdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \dots \\ b_n \end{bmatrix} \tag{4.2}$$

Then, decompose **A**

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ \frac{a_{21}}{a_{11}} & 1 & \dots & 0 \\ \cdots & \dots & \dots & \dots \\ \frac{a_{n1}}{a_{11}} & \frac{a_{n2}}{a_{22}} & \dots & 1 \end{bmatrix} \mathbf{U} \tag{4.3}$$

where **U** is the upper triangular matrix obtained in Eq.4.2 above.

The system of equations **Ax**=**b** is converted to **LUx**=**b**, so that **y**=**Ux**, then **Ly**=**b**, you can find the value of **y**, and from **Ux**=**y**, you can get the value of **x**.

The analytical derivation presented above reveals that the core computational framework of HPL fundamentally relies on matrix transformation principles and essential linear algebra operations, particularly matrix multiplication and inversion. Therefore, it is clearly that the optimization methods for HPL test should focus on the optimization of matrix operations.

4.2. **Performance Optimation Methods.**

4.2.1. *Modification of* `hpl.dat`.

In the data document `hpl.dat` of HPL, many parameters affect the test results of HPL, among which **the most important parameters are the problem size N, the block size NB, and the process array configuration ($P \times Q$)**. Their optimal values are usually determined by factors such as the structure of the CPU, and we can obtain their optimal values for HPL through multiple estimates

and tests. The other parameters have a smaller impact on the results, and we can also determine their optimal values through partial fine-tuning.

(1) **Solving the size of the matrix Ns:** The N value determines the size of the matrix. And the larger the scale N of the matrix, the higher the proportion of effective computation and the higher the system's floating-point processing performance. However, at the same time, the increase in matrix scale N will lead to an increase in memory consumption. Once the actual memory space of the system is insufficient, using the cache will significantly reduce performance. Therefore, for general systems, it is necessary to try to increase the scale N of the matrix while ensuring that the system cache is not used. Considering that the operating system itself requires a certain amount of memory, in addition to matrix A ($N \times N$), HPL also has other memory overheads, and communication also needs to occupy some cache (the specific size of which varies depending on different MPIs). Generally, matrix A occupying about 80% of the total system memory is optimal $N \times N \times 8$=total system memory $\times$80%. We made a series of experiences as shown in Figure 5. From the Figure 5, we can conclude that the Ns on our Machine should be about 86000.

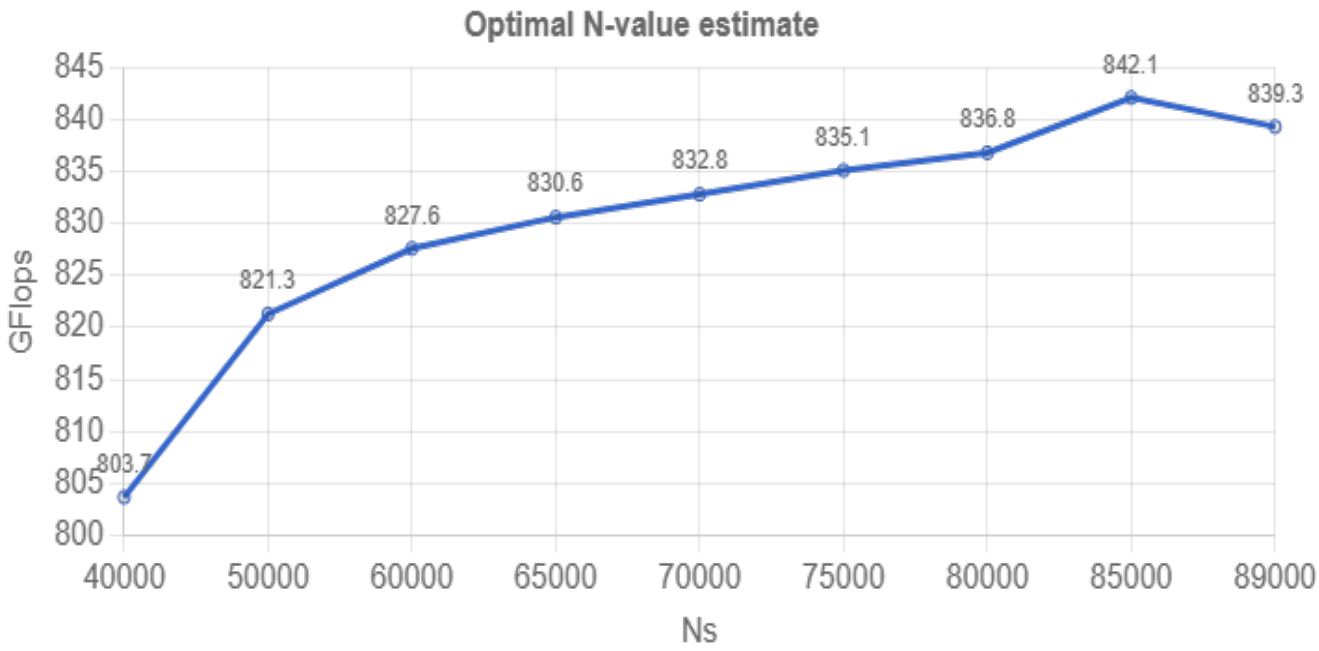


Figure 5. *Optimal N value estimation for HPL test*

(2) **Solving the size of the matrix block(NBs):** NB represents the size of matrix block division. The size of the calculation granularity NB very much affects the pros and cons of the calculation performance. The coefficient matrix is divided into cyclic blocks of $NB \times NB$ and assigned to each process Medium to medium processing, too coarse or too fine particles will affect performance. Because the communication between cluster nodes adopts the message passing method, the NB is too small to pass messages frequently between

nodes, resulting in long communication time, short computing time, and low computing/-communication performance ratio; The NB is too large, which can easily lead to uneven load, which will also increase communication overhead. Therefore, communication between and within nodes also requires trade-offs. The selection of NB value is mainly to obtain the optimal value through actual testing. According to the test, NB should not be too large or too small, generally below 512, generally a multiple of 64. After multiple tests, we plotted the test results into Figure 6. As can be seen from Figure 6, when the value of NBs=384 on the machine we tested, we can obtain better results.

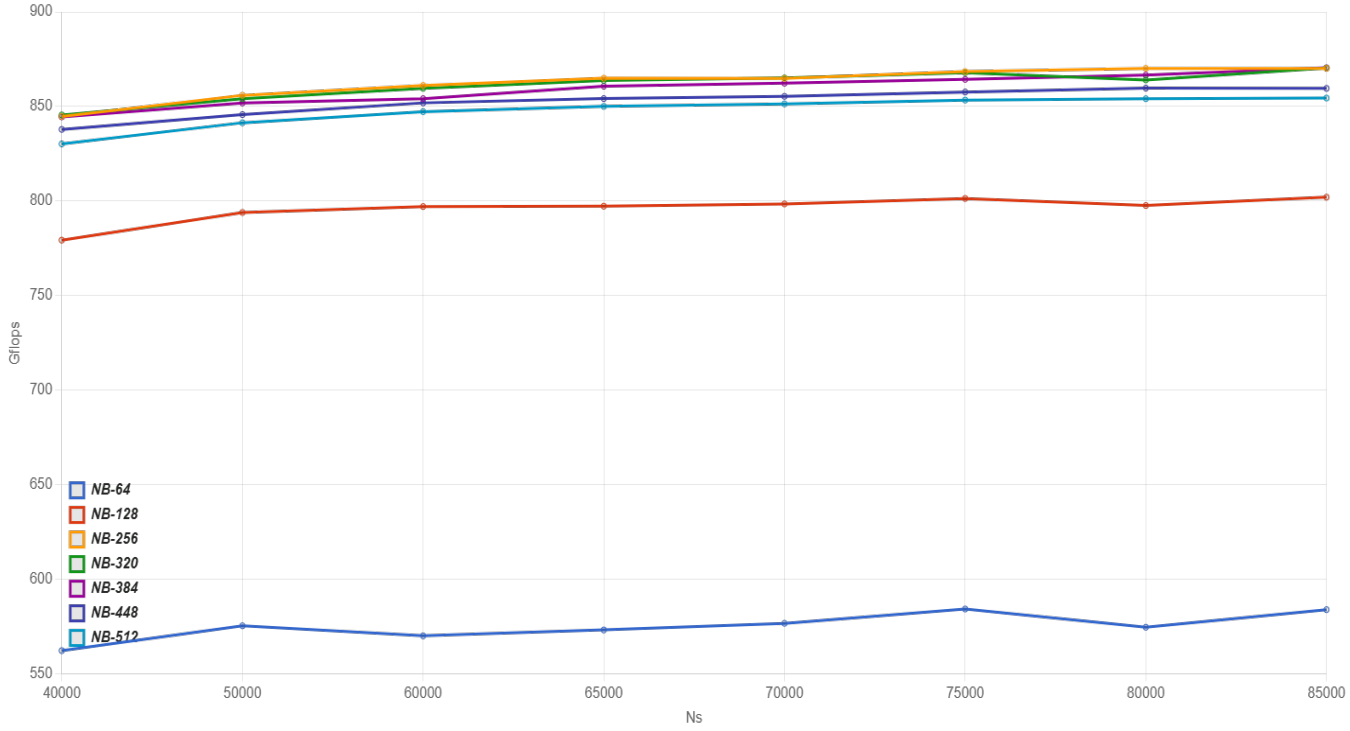


Figure 6. *Optimal NB value estimation for HPL test*

(3) **Two-dimensional processor grid ($P\times Q$):** ($P\times Q$) represents the process grid, which is equivalent to the number of processors in the cluster. When choosing these two numbers, try to make the grid as "square-shape" as possible in shape. As stated on the HPL website, the best practice is to make the grid as close to a "squar" as possible, that is, P and Q should be approximately equal and Q should be slightly larger than P.
Therefore, the two-dimensional processor grid ($P\times Q$) should best meet the following requirements:
$P\times Q$ = the number of processes;
$P\times Q$ = the number of GPUs = the number of processes.
One process corresponds to one GPU, and a process can spawn multiple threads to saturate the CPU.
$P \leq Q$.
Through multiple tests, the program is run with the number of threads=16, and it is found that when the values of P and Q are 4, 4 or 2, 8, the running results show better.

4.2.2. *Third-Party Library.*

There are two alternative third-party libraries: the MPI library and the BLAS library. Commonly used MPI libraries include MPICH, IMPI, and OpenMPI. BLAS libraries include CBLAS, MKL, OpenBLAS, and more. Most of the computation on the CPU is performed by the BLAS library, so the BLAS library has a relatively large impact on the computation. Therefore, it is still very important to choose the right BLAS library. Based on the paper and competition materials from previous years, we initially chose OpenBLAS and MKL as the basic linear algebra subroutines, followed by MPICH and OpenMP as the MPI libraries for testing, with the following results:

Table 4. *Comparison of different BLAS libraries and compilers for HPL test*

Configuration	**Performance**
OpenBLAS + Intel Compiler	345.9 GFlops
MKL + Intel Compiler	**659.3 GFlops**
OpenBLAS + GNU Compiler	341.2 GFlops
MKL + GNU Compiler	623.7 GFlops

As can be seen from Table 4, even after configuring with various third-party libraries, the performance is still poor. Considering that we are using an AMD processor and the execution results of the Intel series libraries are not satisfactory when used on AMD processors, we used the AOCC provided by AMD's official website as the compiler and the AMD BLAS library released by it. Based on the binary files released by AMD's official website for testing, the GFlops test results of HPL have significantly improved, as shown in Table 5. Utilizing AMD's libraries enables the performance test to reach 868.99 GFlops, which is 209.69 GFlops higher than the best Intel test result.

Table 5. *AMD's BLAS library and compiler for HPL test*

Configuration	**Performance**
AOCC + AOCL	**868.99 GFlops**

4.2.3. *Accelerate Procedures by Using Appropriate Compiler Options.*

In order to better achieve high-performance optimization, we have debugged the compilation options of the compiler(shown in Table 6), which helps to make full use of the compiler's computing power and can provide some necessary links for the calculation.

Table 6. *Compile flags*

Compile Options	Functional Description
-fomit-frame-pointer	Omit frame pointer
-O3	Enable all conventional optimizations
-funroll-loops	Enable loop unrolling optimization
-m64	Specify compiler to generate 64-bit code
-Wl,–start-group -Wl,–end-group	Used for linkers
-lpthread	Add a pthread library at link time
-lm	Include the Math Library when linking

4.2.4. *Multi-thread acceleration.*

Based on the HPL testing algorithm, the matrix is decomposed into **LU** decomposition by dividing it into blocks using multiple processes. Each process is responsible for a matrix block, and OpenBLAS indeed supports multi-threading acceleration, and the number of threads used can be controlled through environment variables. Through reasonable thread configuration and parallel computation, OpenBLAS can significantly improve the performance of linear algebra calculations, especially for large-scale matrix operations; at the same time, OpenMP also supports multi-threading acceleration, which is particularly suitable for parallel computation on shared memory architectures. It is a parallel programming model for compiler instructions and runtime environments, allowing programs to execute tasks in parallel on multiple processor cores, thereby improving computational efficiency.

So we guess that if multi-threads is used to accelerate matrix calculations, multi-threads for computation can improve performance. Enhance scalability, optimize memory and cache access, help with load balancing, and improve running efficiency.

4.3. **Machine Performance Analysis.**

Table 7. *Machine hardware configuration*

Name	Model	Quantity
CPU	AMD EPYC 9124 16-Core Processor	1
GPU	NVIDIA GeForce RTX 4070 Ti Super	2
RAM	DDR	128G
Hard Disk	SSD	1T

According to the above configuration, we can calculate the theoretical peak performance for CPU and GPU as follows:

(1) **The overall CPU theoretical peak performance is :**

$$3.7GHz\times16(cores/CPU)\times1(CPUs)\times16(floatingpointexecutionunit)=947.2GFlops$$

(2) **The theoretical peak performance of one GPU is :**

$$689GFlops\times1 = 689GFlops$$

4.4. **Experiment Results.**

4.4.1. *CPU Evaluation Results.*

We use the CPU to run HPL. To achieve the best performance, we tried different combinations of values of N, NB, and PXQ in the configuration file `hpl.dat` . The final parameters in `hpl.dat` are as shown in Figure 7 with N=85000, NB=384,$P\times Q$=1×1.

Based on the above optimization ideas, we also performed several experiments by combining different libraries and compilers. Table 8 shows the evaluation results.

Finally, we can obtain the maximum peak efficiency:

$$\text{Efficiency} = \frac{\text{Peak of HPL}}{\text{Theoretical Peak}} = \frac{868.99\,\text{GFlops}}{947.2\,\text{GFlops}}=91.74\%$$

```
HPLinpack benchmark input file
Innovative Computing Laboratory, University of Tennessee
HPL.out      output file name (if any)
6            device out (6=stdout,7=stderr,file)
1            # of problems sizes (N)
85000 Ns
1            # of NBs
384       NBs
0            PMAP process mapping (0=Row-,1=Column-major)
1            # of process grids (P x Q)
1       Ps
1       Qs
16.0         threshold
1            # of panel fact
1            PFACTs (0=left, 1=Crout, 2=Right)
1            # of recursive stopping criterium
48           NBMINs (>= 1)
1            # of panels in recursion
8            NDIVs
1            # of recursive panel fact.
2            RFACTs (0=left, 1=Crout, 2=Right)
1            # of broadcast
1            BCASTs (0=1rg,1=1rM,2=2rg,3=2rM,4=Lng,5=LnM)
1            # of lookahead depth
0            DEPTHs (>=0)
1            SWAP (0=bin-exch,1=long,2=mix)
64           swapping threshold
0            L1 in (0=transposed,1=no-transposed) form
0            U   in (0=transposed,1=no-transposed) form
0            Equilibration (0=no,1=yes)
8            memory alignment in double (> 0)
```

Figure 7. *`hpl.dat` of CPU HPL test*

Table 8. *Best performance of CPU HPL test*

Configuration	Performance
MKL+INTEL Compiler	659.28 GFlops
OPENBLAS+INTEL Complier	345.9 GFlops
AOCC+AMDBlis+AOCL	**868.99 GFlops**

4.4.2. *GPU Evaluation Results.*

First, we configured the HPL test of the GPU based on experience and found that our test results were not good. Then considering that we are using AMD processors, we decided to learn the binary packages that AMD uses. Figure 8 shows the best profile for the performance test with N=34816, NB=1024, $P\times Q$=1×1., and Figure 9 shows the performance results.

Finally, we can get the efficiency:

$$\text{Efficiency} = \frac{\text{Peak of HPL}}{\text{Theoretical Peak}} = \frac{631.2\,\text{GFlops}}{689\,\text{GFlops}} = 91.6\%$$

4.5. **Summary.**

To sum up, we can see that in HPL testing, in addition parameters N, NB, $P\times Q$, affecting the test results, different BLAS libraries and compilers perform differently on specific hardware. In the end, **our CPU efficiency reached 91.74% and GPU efficiency reached 91.6%**.

```
HPLinpack benchmark input file
Innovative Computing Laboratory, University of Tennessee
HPL.out      output file name (if any)
6            device out (6=stdout,7=stderr,file)
1            # of problems sizes (N)
34816 Ns
1            # of NBs
1024    NBs
1            PMAP process mapping (0=Row-,1=Column-major)
1            # of process grids (P x Q)
1            Ps
1            Qs
16.0         threshold
1            # of panel fact
0            PFACTs (0=left, 1=Crout, 2=Right)
1            # of recursive stopping criterium
2            NBMINs (>= 1)
1            # of panels in recursion
2            NDIVs
1            # of recursive panel fact.
0            RFACTs (0=left, 1=Crout, 2=Right)
1            # of broadcast
4            BCASTs (0=1rg,1=1rM,2=2rg,3=2rM,4=Lng,5=LnM)
1            # of lookahead depth
1            DEPTHs (>=0)
1            SWAP (0=bin-exch,1=long,2=mix)
64           swapping threshold
1            L1 in (0=transposed,1=no-transposed) form
0            U    in (0=transposed,1=no-transposed) form
0            Equilibration (0=no,1=yes)
8            memory alignment in double (> 0)
```

Figure 8. *hpl.dat of GPU HPL test*

```
================================================================================
=
T/V                N    NB     P     Q               Time                 Gflops (     per
GPU)
--------------------------------------------------------------------------------
WC0             34816  1024     1     1              44.58          6.312e+02 ( 6.312e+02)

HPL_pdgesv() start time Thu Jan 16 18:57:09 2025
HPL_pdgesv() end time   Thu Jan 16 18:57:54 2025

--------------------------------------------------------------------------------
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)=        0.000487153493 ..... PASSED
```

Figure 9. *Performance result of GPU HPL test*

5. HPCG Test and Optimization

This section focuses on the examination and evaluation of HPCG. All the test outcomes presented in this study are derived from the same hardware platform utilized for HPL, as detailed in Table 7.

5.1. **Analysis of HPCG.**

The High Performance Conjugate Gradient (HPCG) benchmark is a key tool for assessing the performance of high-performance computing (HPC) systems, particularly in simulating real-world computational workloads. Unlike traditional benchmarks such as HPL, which mainly measure floating-point operations through dense matrix multiplication, HPCG focuses on solving large sparse linear systems. These systems are derived from the discretization of a three-dimensional partial differential equation (PDE) that simulates a heat diffusion problem with zero Dirichlet boundary conditions.

At the heart of HPCG is the preconditioned conjugate gradient (PCG) method, which necessitates efficient handling of sparse matrix operations, optimized memory bandwidth, and parallel computation across distributed systems. The global problem is defined on a three-dimensional grid with dimensions $(nx \times npx) \times (ny \times npy) \times (nz \times npz)$, where nx, ny, and nz represent the local subgrid sizes in the x, y, and z dimensions, respectively, assigned to each MPI process. The values npx, npy, and npz denote the number of processes in each corresponding dimension. These values are automatically computed during the HPCG setup phase but can also be manually specified in the input file `hpcg.dat` or provided as command-line arguments.

The HPCG benchmark consists of three critical computational kernels, each integral to evaluating modern supercomputer architectures:

(1) Sparse Matrix-Vector Multiplication (SpMV): The core computational task of HPCG, SpMV is computationally intensive and central to the benchmark's performance evaluation.
(2) Symmetric Gauss-Seidel (SymGS) Relaxation: This serves as the essential preconditioning technique for the conjugate gradient method, enhancing the efficiency of the iterative solution process.
(3) Conjugate Gradient Method: The iterative approach used to solve the large-scale sparse linear systems, making it a crucial component of HPCG's overall performance.

These kernels are commonly found in scientific computing applications, such as computational fluid dynamics and electromagnetic simulations. They present significant challenges for system memory bandwidth and fine-grained parallel efficiency, making them indispensable for benchmarking modern HPC architectures.

5.2. **Performance Optimation Methods.**

5.2.1. *Modification of HPCG.dat.*

By modifying the configuration parameters in the `HPCG.dat` file, the optimization of the standard HPCG benchmark can be significantly impacted. The file contains two key sections that can be adjusted to improve performance: the problem size (specifically, the matrix size) and the runtime. Adjusting the matrix size can have a major effect on performance.

(1) **Matrix Size:** The matrix size defines the scale of the problem and is directly related to the global dimensions of the discretized computation grid. By adjusting the local subgrid dimensions (nx, ny, nz) and the number of subgrids in each dimension (npx, npy, npz), the workload assigned to each MPI process can be optimized. A larger matrix size generally increases computational complexity, which improves the scalability and performance of the benchmark on a large number of processors. However, excessively large problem sizes

may lead to memory limitations or imbalanced workload distribution between processes. Therefore, careful consideration of these parameters is essential for achieving optimal performance.

Fig.10 presents the HPCG performance results based on the standard HPCG benchmark with a fixed runtime of 1800 seconds across varying matrix sizes. **SpMV** represents the performance of Sparse Matrix-Vector Multiplication (SpMV), a core computational task in the benchmark, measured in GFLOP/s. **MG** reflects the performance of the Multigrid (MG) algorithm, an important preconditioning technique for solving the linear system, also measured in GFLOP/s. **Total** aggregates the performance of all benchmark tasks, providing an overall computational throughput. Finally, the **Final Summary** provides the final GFLOP/s rating and confirms the validity of the test results.

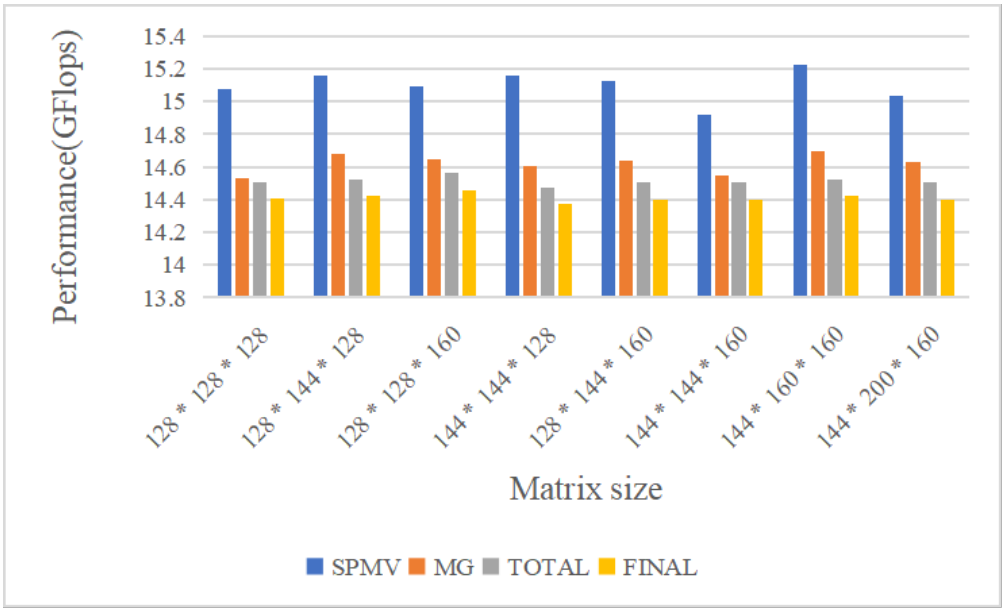


Figure 10. *Performance results for different matrix size settings in standard HPCG Benchmark*

(2) **Runtime:** The runtime is another important parameter that influences the performance of the HPCG benchmark. A longer runtime allows for more thorough testing of the system's ability to handle sustained computational load. While a longer runtime provides a better measure of the system's long-term stability and computing capacity, excessively long runtimes may lead to diminishing returns in performance due to hardware bottlenecks. Therefore, choosing an appropriate runtime is essential to strike a balance between accuracy and benchmark duration. In addition to the size of the topic scale, the shortest running time should be set to ensure the long enough running time of the system.

Optimizing the matrix size and runtime in the `HPCG.dat` file is crucial for fine-tuning benchmark performance, ensuring that the results reflect the system's true capabilities. These modifications help with better load distribution, reduced memory overhead, and improved overall efficiency in parallel computing environments. Figure 11 presents the performance results based on the standard HPCG benchmark, conducted under varying runtime conditions, with the matrix size setting as $(nx \times ny \times nz) = (128 \times 128 \times 160)$ corresponding to the optimal parameter setting in Figure 10.

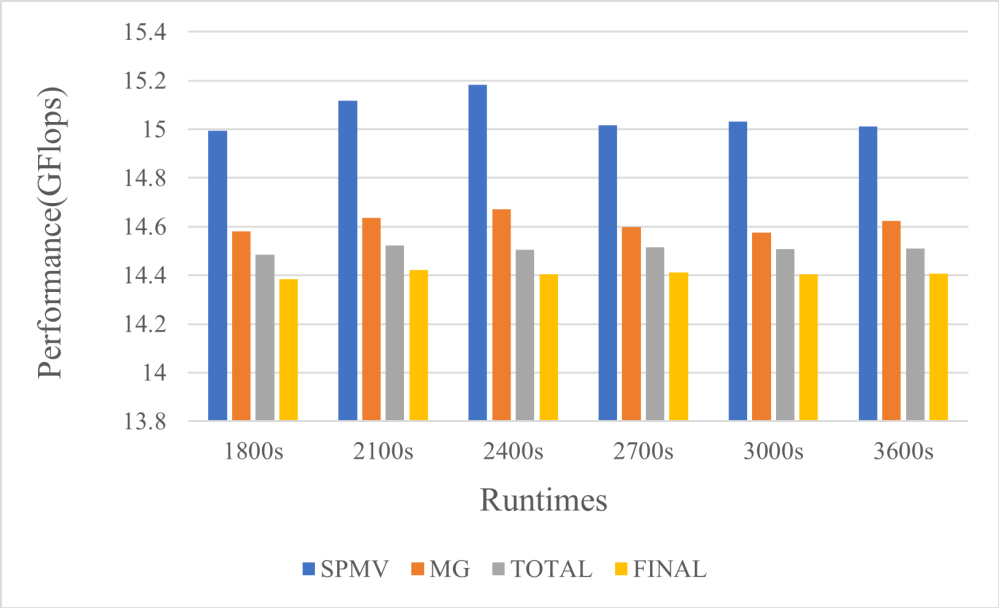


Figure 11. *Performance results for different runtime settings in standard HPCG Benchmark*

5.2.2. *Third-Party Library for HPCG.*

For the standard HPCG benchmark, there are several third-party libraries available that can enhance performance. Key libraries include those for MPI (Message Passing Interface) and BLAS (Basic Linear Algebra Subprograms).

The commonly used MPI libraries are MPICH, OpenMPI, and Intel MPI (IMPI). These libraries handle the communication between different processes in a distributed computing environment. For BLAS, several implementations are available, such as MKL (Intel Math Kernel Library), OpenBLAS, and Atlas. BLAS libraries are crucial for performing optimized linear algebra operations on the CPU, which form a significant part of the HPCG computation.

After testing, we chose OpenBLAS and MKL as the BLAS libraries, while OpenMPI and Intel MPI were selected as the MPI libraries for communication. The matrix size used in our experiments is set to $(160 \times 160 \times 128)$, which corresponds to the optimal parameter setting in Figure 10. The runtime is fixed at 1800 seconds, which is the minimum runtime specified by the standard HPCG benchmark.

After ensuring the proper installation and configuration of the aforementioned third-party libraries, we use both the GNU Compiler and the Intel Compiler to compile, link, and utilize these libraries by setting the appropriate compiler flags. The performance results from these configurations are presented in Table 9. Figure 12 provides a more intuitive display of the results.

Table 9. *Performance Comparison for Different Machines and BLAS Libraries*

Configuration	Compiler	Performance (GFlops)
Openmpi + Openblas	GNU Compiler	14.4596
Openmpi + Openblas	Intel Compiler	14.4241
Openmpi + MKL	Intel Compiler	14.4294
Intelmpi + MKL	Intel Compiler	**14.4639**
Intelmpi + MKL	GNU Compiler	14.3838

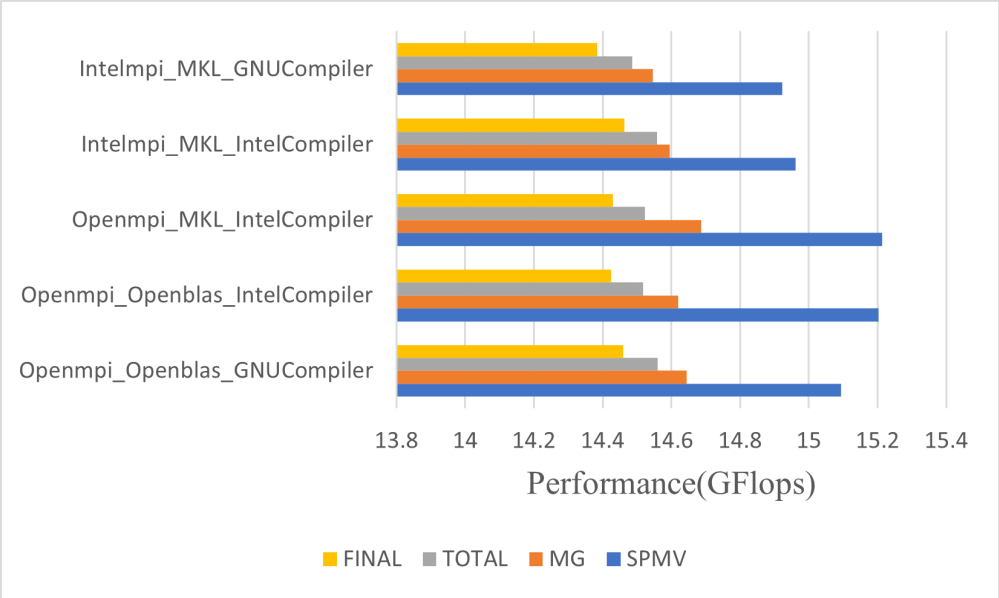


Figure 12. *Performance results for different third-party library settings in standard HPCG Benchmark*

As seen from the results, using Intel Intelmpi, MKL with the Intel Compiler achieved the highest performance among all tested configurations. This indicates that the optimized Intel libraries provide the best performance on the CPU for the standard HPCG benchmark.

5.2.3. *Parallelization Paradigms for HPCG.*

In this section, we explore three different configurations for testing the standard HPCG benchmark: using MPI alone, using OpenMP alone, and using both MPI and OpenMP together. These configurations allow us to compare the performance of the standard HPCG benchmark under different parallelization paradigms and understand the impact of each parallelization method.

The matrix size used in our experiments is set to $(160 \times 160 \times 128)$, with the runtime fixed at 1800 seconds. The third-party libraries used are OpenMPI and OpenBLAS, and the GNU Compiler is employed for compilation.

(1) **HPCG with MPI**

When running the standard HPCG benchmark with MPI alone, the computation is parallelized across multiple processes, with each process handling a portion of the matrix. MPI handles the communication between processes, ensuring that data is exchanged correctly between different parts of the grid. This configuration is ideal for distributed memory systems, where each node or process has its own memory and communicates over a network.

The MPI-only version of HPCG can efficiently utilize multiple processors across a cluster, but the communication overhead between processes may limit performance as the number of processes increases.

(2) **HPCG with OpenMP**

In contrast, using OpenMP alone focuses on parallelizing the computation within a single node, where multiple threads are used to take advantage of multi-core CPUs. OpenMP uses shared memory within a node, allowing threads to access common data structures without the need for explicit communication between them. This configuration is particularly

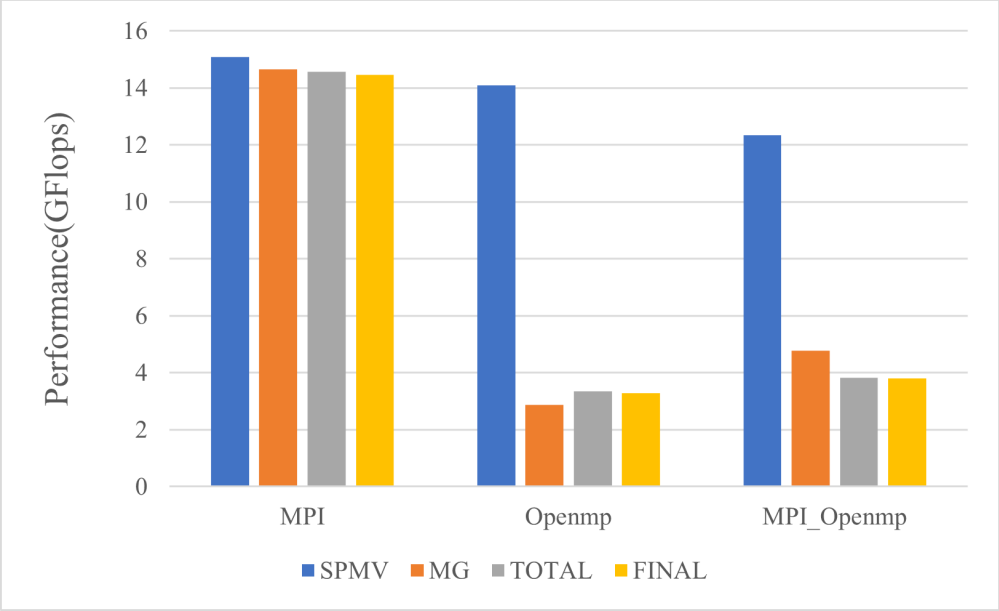


Figure 13. *Performance results for different Parallelization Paradigm settings in standard HPCG Benchmark*

suitable for systems with multiple cores on a single node and can effectively speed up the computation by exploiting thread-level parallelism.

While OpenMP is highly effective on multi-core processors, its performance may be limited by the memory bandwidth and CPU core utilization, especially when dealing with large-scale problems.

(3) **HPCG with MPI and OpenMP (Hybrid)**

A hybrid approach that combines MPI and OpenMP allows the standard HPCG benchmark to take full advantage of both distributed memory and shared memory parallelism. In this configuration, MPI is used for inter-process communication across nodes, while OpenMP is used to parallelize computation within each node. This approach provides better scalability on multi-node systems, as it can efficiently utilize both multiple nodes (via MPI) and multiple cores within each node (via OpenMP).

The hybrid MPI+OpenMP configuration is ideal for modern HPC systems, which often have many multi-core nodes connected via high-performance networks. By combining both parallelization models, this approach can deliver the best performance, provided that the problem size is sufficiently large and the load balancing between MPI and OpenMP is optimized.

(4) **Performance Comparison and Analysis**

Based on the results of the experiments, as shown in Figure 13, we observed that the configuration using **MPI alone** achieved the best performance across all tested machines. This configuration allowed the standard HPCG benchmark to fully utilize the distributed memory capabilities of the system, with each MPI process handling a portion of the problem. The communication overhead, though present, was minimized due to the high efficiency of MPI in handling large-scale parallel problems.

In contrast, using **OpenMP alone** resulted in lower performance, primarily due to the limited scalability on a single-node setup. Although OpenMP effectively leveraged multi-core CPUs, it was constrained by memory bandwidth and the relatively limited computational power available within a single node.

The **hybrid MPI+OpenMP** configuration performed well on multi-node systems, but the performance improvement was not as significant as expected. This is likely due to sub-optimal load balancing between MPI and OpenMP, where the communication overhead between processes overshadowed the benefits of multi-threading within nodes. Moreover, hybrid configurations often require fine-tuning to achieve maximum performance, especially with larger problem sizes, which can result in a less straightforward optimization process.

5.2.4. *Hybrid Version of HPCG.*

To further optimize performance, a hybrid approach can be applied to the HPCG benchmark, integrating CUDA for GPU acceleration. This allows the GPU's floating-point performance to be exploited without altering the basic conjugate gradient (CG) algorithm or preconditioner algorithm. The GPU's parallel processing power is harnessed to significantly speed up matrix operations, which are a critical part of the HPCG benchmark.

In this hybrid version, the matrix is analyzed, reordered, and the computational workload is split between the CPU and GPU. The CPU handles portions of the matrix that are better suited for traditional processors, while the GPU is responsible for the portions that benefit most from parallel computation capabilities. This approach allows the hybrid version to capitalize on the strengths of both the CPU and GPU, optimizing performance for large-scale computations.

Fine-tuning the ratio of CPU to GPU workload is essential to achieve optimal performance. By experimenting with different configurations and workloads, the ideal distribution of tasks between CPU and GPU can be found, maximizing the overall performance of the HPCG benchmark on systems equipped with both CPU and GPU resources.

The hybrid version allows HPCG to fully utilize both CPU and GPU resources, providing an effective solution for high-performance computing systems with multi-core CPUs and GPUs.

5.3. **Experimental Results.**

5.3.1. *CPU Evaluation Results.*

Since our server uses an AMD CPU, using the AMD-specific HPCG benchmark allows for the full utilization of its hardware features, resulting in a more accurate performance evaluation. Compared to the standard HPCG benchmark, the AMD version is optimized to better reflect performance in real-world applications. The following describes the test experiments for the AMD-specific HPCG benchmark.

First, We conducted experiments by using only the CPU to run the AMD-specific HPCG benchmark. To achieve the best performance, we tested different data scales by adjusting the matrix size in the configuration file `HPCG.dat`. By fine-tuning the matrix size, we sought to identify the optimal configuration for CPU-based computations on our AMD hardware.

The results of different parameter settings are shown in the figure 14.

After testing multiple configurations, the optimal performance achieved was 21.1145 GFlops with the final parameters in the `HPCG.dat` file as follows:

- Matrix Size: $128 \times 128 \times 160$
- runtime: 1800s
- Number of subgrids: npx=4, npy=2, npz=2

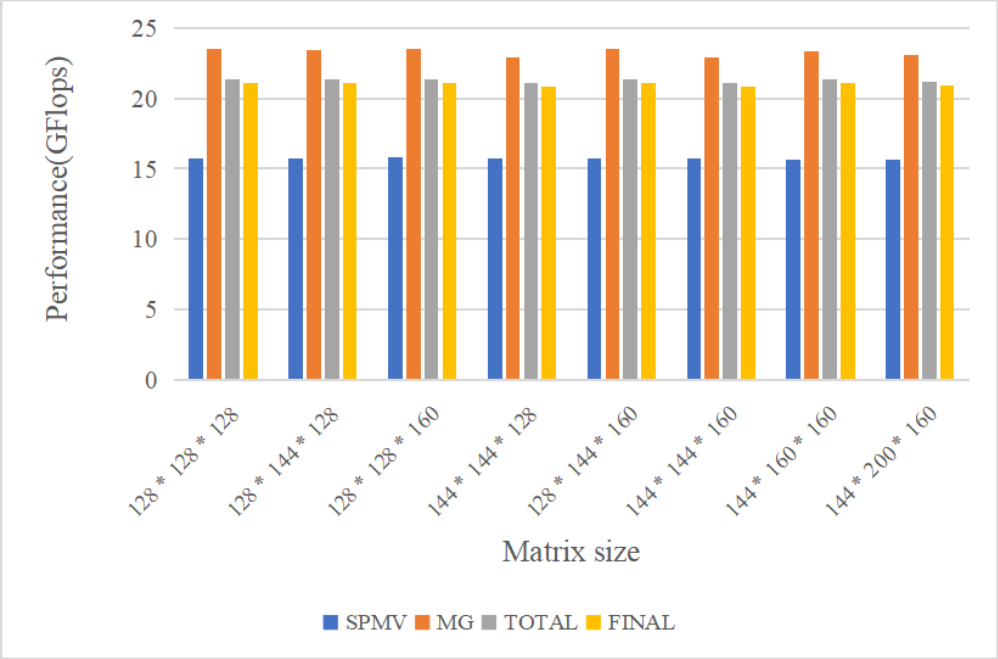


Figure 14. *Performance results for different parameter size settings in AMD HPCG Benchmark*

These parameters were selected after thorough testing to balance computational complexity and memory usage while ensuring that the system operated efficiently under the CPU-only configuration.

Finally, the computational efficiency is quantified as the ratio of the peak performance of HPCG to theoretical peak of our machine, which is calculated as:

$$\text{Efficiency} = \frac{\text{Peak of HPCG}}{\text{Theoretical Peak}} = \frac{21.1145\,\text{GFlops}}{947.2\,\text{GFlops}} = 2.23\%$$

It can be seen that the peak efficiency of HPCG(2.23%) is inferior to that of HPL(91.73%). This is because HPCG is more closely aligned with the sparse matrix operations in practical applications rather than dense matrix multiplications.

5.3.2. *GPU Evaluation Results.*

For GPU evaluation, we utilized the CUDA-enabled version of the HPCG benchmark.

The version we used is provided by NVIDIA and cannot be modified. Although we attempted to analyze the source code as time permitted, it is undeniable that there is a significant gap between our understanding and the optimizations implemented in the NVIDIA-provided version. Therefore, our current optimizations for HPCG were primarily achieved through parameter tuning in the `HPCG.dat` file. These adjustments contributed to improved performance on GPU-based computations.

Of course, we have two GPUs (Tesla P100 PCIe 16GB), and we were able to utilize both GPUs simultaneously to maximize efficiency. By enabling multi-GPU execution, we aimed to explore the impact of distributed computation across multiple GPUs and achieve higher performance.

As shown in Table 10, we have tested the best performance of single GPU and dual GPU.

The results show a significant performance improvement when using dual GPUs compared to a single GPU for the HPCG benchmark. The dual GPU configuration achieves 129.278 GFlops, nearly doubling the performance of the single GPU configuration, which reaches 255.55 GFlops.

Table 10. *GPU-based Performance Results for HPCG*

Configuration	Performance (GFlops)
Single GPU	129.278 GFlops
Dual GPU	255.55 GFlops

This improvement can be attributed to several factors: parallelization across GPUs, which effectively doubles computational resources and enhances throughput; increased memory bandwidth, as each GPU handles a portion of the memory, reducing bottlenecks; and the increased computational power from using two GPUs in parallel. Additionally, optimized multi-GPU execution allows for efficient workload distribution, further accelerating computation. Overall, the dual GPU setup outperforms the single GPU by leveraging parallelism, memory bandwidth, and computational power, highlighting the advantages of scaling computations for high-performance tasks like the HPCG benchmark.

The results of different parameter settings for dual GPU-based computations are shown in the figure 15.

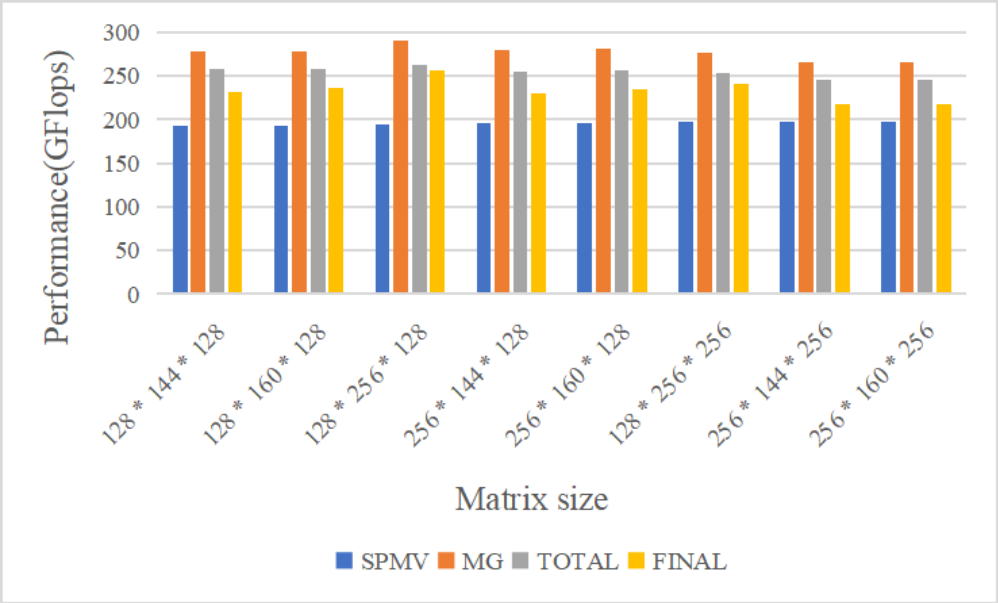


Figure 15. *Performance results for different parameter settings on dual GPU-based HPCG Benchmark*

After testing multiple configurations, the optimal performance achieved was 255.55 GFlops, the final parameters in the `HPCG.dat` file for GPU-based computations were as follows:

- Matrix Size: $128 \times 256 \times 128$
- runtime: 1800s
- Number of subgrids: npx=1, npy=1, npz=2
- Number of GPUs: 2 (Dual GPU configuration)

These parameters were selected after thorough testing to balance computational complexity and memory usage while ensuring that the system operated efficiently under the dual GPU configuration.

Finally, the computational efficiency is expressed as:

$$\text{Efficiency} = \frac{\text{Peak of HPCG}}{\text{Theoretical Peak}} = \frac{255.55\,\text{GFlops}}{689\,\text{GFlops} \times 2} = 18.54\%$$

It is also less efficient than HPL and conforms to the characteristics of sparse matrix operations.

5.4. **Summary.**

5.4.1. *CPU Version.*

(1) **Efficiency Observations** In the HPCG tests, our **CPU efficiency reached as 21.1145 GFlops with 2.23% efficiency.**

(2) **Optimization Plans:**

(a) **Memory Upgrade**: The next step will involve increasing the memory of Server. However, this optimization cannot be carried out immediately due to some unavoidable constraints.

(b) **Performance Tools**: To better understand and optimize the performance, the plan is to utilize performance analysis tools like `TVune` and `Gprof`.

- `TVune` is a track-based profiler that can capture a comprehensive set of performance metrics, including CPU calculations, GPU utilization, bandwidth, thread scheduling, and more. These data will be visualized to enhance clarity.
- `Gprof` will be used to track function call relationships and the number of calls per function, providing further insights into potential inefficiencies.

5.4.2. *GPU Version.*

(1) **Efficiency Observations:**

In the HPCG tests, the GPU outperforms the CPU significantly in terms of efficiency. **The GPU's computational peak reaches 255.55 GFlops with 18.54% efficiency**, whereas the CPU achieves only 21.1145 Gflops with 2.23% efficiency.

(2) **Optimization Plans:**

(a) **Communication Optimization Between GPUs**: The next optimization step involves addressing the communication challenges between GPUs. The goal is to improve multi-GPU performance and enable HPCG computations with multiple GPUs.

(b) **Increasing GPU Count**: Increasing the number of GPUs within a single server is another step in optimizing GPU utilization for higher performance.

Given the relative inefficiency observed in the CPU-only version, integrating CPU and GPU resources into a hybrid configuration presents a promising strategy to enhance performance in high-performance computing (HPC) benchmarks like HPCG. The parallel processing capabilities of GPUs, combined with the versatility of CPUs, offer a powerful approach to handling large-scale, computationally intensive problems.

6. Optimization for AlphaFold3 Inference

6.1. **Introduction.**

AlphaFold 3, the latest generation of protein structure prediction models developed by DeepMind, represents a significant advancement over AlphaFold 2 [5]. It is not only capable of predicting the three-dimensional structures of single-chain proteins with remarkable accuracy but also adept at handling complex biological scenarios, including multi-chain complexes, ligand binding, and dynamic conformational changes. This breakthrough provides a powerful tool for biomedical research, drug design, and fundamental scientific exploration. However, the inference process of AlphaFold 3 is highly computationally intensive, particularly when dealing with large-scale datasets or long sequences.

In this competition, our primary focus was on minimizing the inference time of the AlphaFold 3 GPU model. We also aimed to migrate the model to a CPU-based system while minimizing the runtime. On the GPU side, we optimized performance through techniques such as data parallelism and tensor sharding, fully leveraging the computational power of GPUs to enable multi-GPU execution. After migrating the inference codebase to the CPU, we proposed an efficient CPU implementation tailored for resource-constrained environments, incorporating optimizations such as XLA compilation and operator-level improvements.

In this proposal, we have two main objectives. The first is to optimize the inference time of the AlphaFold 3 model on GPUs. The second is to migrate the inference of the GPU-based AlphaFold 3 model to a CPU-based environment and optimize the inference time accordingly.

In the GPU optimization of the AlphaFold 3 model, we employed techniques such as single-GPU data parallelism, tensor sharding, and multi-GPU deployment. Unfortunately, the optimization failed on our machine, and although dual-GPU running was achieved, it was probably due to excessive communication overhead that the optimization was not ideal, with implementation details shown in Section 6.3. For the CPU implementation and optimization, we first migrated the model to the CPU by setting the JAX platform to CPU and fixed a NaN problem in diffusion model. And then we apply optimization methods like XLA backend optimizations and operator optimization, and the detail is shown in Section 6.4. Due to time constraints, we executed the CPU migration version of all the input data provided by the organizing committee on the cloud server. Meanwhile, on the local server, we selected several representative data to conduct tests on both the migration version and the existing optimized version. **By comparing the execution results on the cloud server and the local server, it can be observed that the optimized CPU version has, on average, improved the running efficiency by $1.164\times$ compared to the basic version. The detailed analysis is provided in Section 6.5.3.**

Meanwhile, through system call, CPU occupancy and function callgraph analysis, we have identified additional opportunities for optimization in Section 6.6.

6.2. **Related work.**

The AlphaFold series has revolutionized protein structure prediction, but its high computational demands and long inference times have spurred numerous optimization efforts. Since the inference phase of the AlphaFold 3 model primarily involves the Pairformer module, the Diffusion module, and the Confidence module, we have conducted a comprehensive review of optimization strategies. This includes not only methods tailored for the training and inference of the AlphaFold 3 model but also those specifically targeting the optimization of these individual modules.

Traning Acceleration:

Early work by DeepMind utilized multi-TPU clusters for distributed training; however, inference time optimization was limited to single-device execution [5]. FastFold, proposed by Cheng et al., further reduced AlphaFold's training time from 11 days to 67 hours [6].

Inference Acceleration:

(1) **The Pairformer Module:** The Pairformer module is a cornerstone of AlphaFold 3, designed to capture pairwise interactions between residues in protein sequences. Built upon the transformer architecture, it leverages attention mechanisms to model long-range dependencies and refine structural predictions. Despite its effectiveness, the computational demands of this module have motivated extensive optimization efforts.

Su et al. proposed a *dynamic batch sizing method* for multi-chain protein inference, which dynamically adjusts batch sizes based on input complexity [7]. While this approach improves GPU memory utilization, challenges such as memory fragmentation remain unresolved. To address scalability, *Megatron-LM-style parallelism* has been explored, achieving linear scaling across multiple GPUs for large models [8]. However, small-batch scenarios—common in protein folding tasks are often hindered by communication overhead. Additionally, common optimization strategies for large models, such as *data parallelism (DP)*, *distributed data parallelism (DDP)*, and *Zero Redundancy Optimizer (ZeRO)* [9], have been adapted to improve resource efficiency. These techniques, combined with tensor model parallelism, enable the training and inference of larger models while minimizing memory usage. Recent advancements in attention mechanisms, such as *FlashAttention* [10], have further reduced the computational cost of self-attention operations, making the Pairformer module more efficient during inference.

(2) **The Diffusion Module:** The Diffusion module in AlphaFold 3 plays a critical role in iteratively refining predicted protein structures. Inspired by diffusion models widely used in generative modeling, this module gradually denoises an initial noisy structure representation to produce high-quality final predictions. However, the iterative refinement process can be computationally expensive, particularly for large proteins.

To reduce the computational burden, recent work has focused on optimizing sampling techniques. For instance, *Denoising Diffusion Implicit Models (DDIM)* [11] allow for fewer sampling steps while maintaining prediction accuracy. This approach significantly accelerates the inference process without compromising quality. The iterative nature of diffusion models makes them well-suited for parallel execution across multiple GPUs or TPUs. Techniques such as *pipeline parallelism* and *model partitioning* have been successfully applied in other domains [12]. Additionally, leveraging modern hardware accelerators, such as NVIDIA Tensor Cores, through mixed-precision training and inference, further enhances computational efficiency.

(3) **The Confidence Module:** The Confidence module in AlphaFold 3 evaluates the reliability of predicted protein structures by assigning confidence scores to different regions of the structure. These scores are crucial for downstream applications, such as identifying high-confidence regions for experimental validation or guiding further refinement.

Recent efforts have focused on developing lightweight models to approximate confidence scores with minimal computational overhead. For example, *distillation techniques* have been employed to train smaller surrogate models that mimic the behavior of the full confidence module [13]. These approaches not only reduce inference time but also make the module more scalable for large-scale applications.

Building on these insights, we identified opportunities to implement multi-GPU versions for GPU-based optimization and migrated the inference codebase to CPUs, combining data parallelism techniques to minimize the runtime. For the 12 input datasets provided by the organizing committee, we computed results using both GPU and CPU versions of the inference codebase and compared them with the baseline results from the original code. Our optimized implementations achieved confidence levels consistent with expectations.

6.3. **GPU Optimization Methods.**

Our primary objective was to minimize the runtime of the GPU-based code. Below, we first introduce the principles of data parallelism, followed by a discussion of JAX's mechanisms for handling sharding. Finally, by studying the aforementioned concepts of data parallelism and the JAX documentation, we enumerate all our attempts and present the final dual-GPU optimized version.

6.3.1. *Introduction to data parallelism.*

JAX Sharding Techniques

Single Program Multiple Data (SPMD) in JAX is a parallelization technique where the same computation (e.g., the forward pass of a neural network) can be executed in parallel across different devices (such as multiple GPUs) with different input data (e.g., different inputs within a batch).

JAX provides tools such as pjit, xmap, and PartitionSpec to enable efficient tensor sharding across devices. In JAX, each array is associated with a Sharding object that specifies how the tensor is distributed across devices. This mechanism allows for fine-grained control over how computations are partitioned and executed, optimizing both memory usage and computational efficiency. By leveraging these tools, we can achieve scalable and high-performance execution of AlphaFold 3's inference process on multi-GPU setups.

Data Parallelism

Data Parallelism (DP) is a distributed computing technique that splits input data across multiple devices (e.g., GPUs) for parallel processing. Each device independently computes the same model but processes different subsets of the input data. The results are aggregated at the end of each forward pass or layer computation. The overall architecture of data parallelism is illustrated in Figure 16.

A typical data parallel process can be described as follows:

(1) **Computational GPUs and Gradient Aggregation GPU:** As shown in Figure 16, there are several computational GPUs (e.g., GPU0 to GPU2) and one GPU responsible for gradient aggregation, where the AllReduce operation is performed.
(2) **Model Replication:** A complete copy of the model parameters is loaded onto each computational GPU.
(3) **Data Partitioning:** The input data X (e.g., a batch) is evenly distributed among the computational GPUs. Each GPU performs a forward pass (FWD) and backward pass (BWD) on its subset of data, calculating a partial gradient G.
(4) **Gradient Push and Aggregation:** Each computational GPU pushes its computed gradients to the gradient aggregation GPU, where the gradients are combined through an aggregation operation, typically gradient accumulation.
(5) **Gradient Pull and Model Update:** After the aggregation is complete, the computational GPUs pull the aggregated gradients from the gradient aggregation GPU to update their local model parameters W. Once the update is complete, the model parameters on all computational GPUs remain consistent.

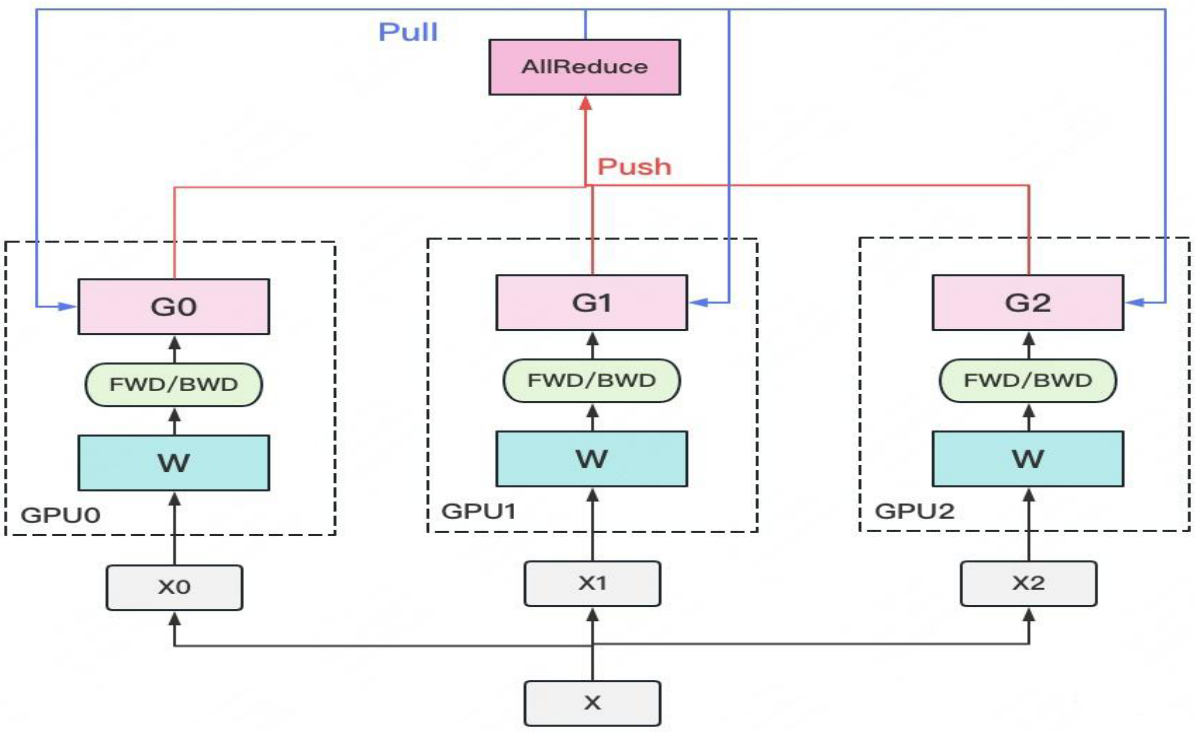


Figure 16. *Overall Architecture of Data Parallelism*

(6) **AllReduce Operation:** The process of aggregating gradients and distributing the results back to the computational GPUs is referred to as the AllReduce operation. This ensures synchronization of model parameters across all devices while maintaining high computational efficiency.

6.3.2. *Data parallelism on dual GPUs.*

(1) Performance Analysis and Identification of Optimization Points

We first executed the Baseline implementation and utilized performance analysis tools to gen-

Figure 17. *Baseline Flame Graph of GPU inference*

erate a flame graph, as shown in Figure 17. Upon observation, **we found that the *PairFormer***

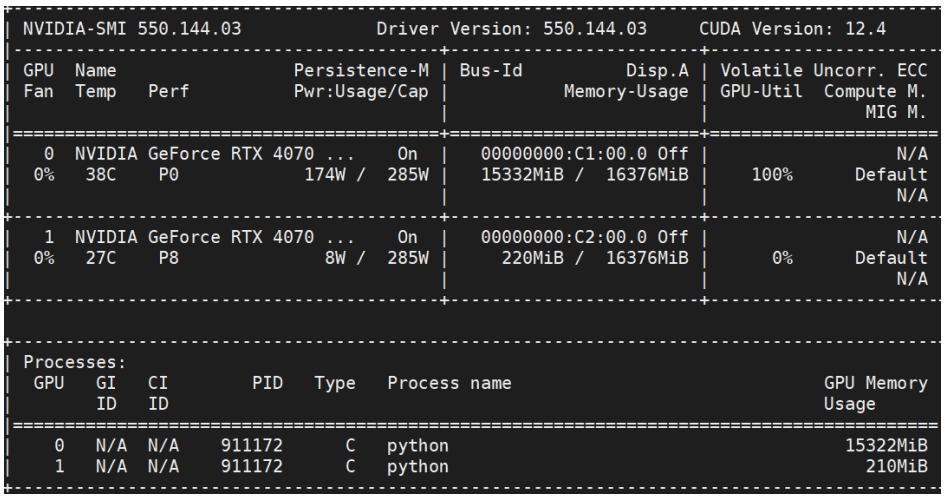


Figure 18. *GPU Utilization of the Baseline Implementation*

processing and the *Diffusion* module consumed a significant amount of time during the inference phase of the AF3 model. Further investigation using GPU monitoring tools revealed that only one GPU was actively utilized during execution, as illustrated in Figure 18.

Through detailed analysis, we identified that the *Evoformer* class was being called multiple times in a loop, and its parallelization potential had not been fully realized. Based on these findings, we decided to parallelize the *PairFormer* module by leveraging JAX's data sharding capabilities to achieve data parallelism.

(2)Parallel Optimization of Pairformer

In the run_inference function in the run_alphafold.py file, the part of the call self._model is processed in parallel, and the input data featurised_example is sharded to two GPUs, but the effect is not good, and the communication overhead between devices may be too large due to the large amount of data.

For the parallel optimization of the calculation body part in the diffusion_head.py file, the parallelism error occurs due to the incompatibility of the PMAP parallelism of the JAX and the Haiku library. After using shard_map, the shape does not match for subsequent steps, and there is no optimization success.

Here is the core code for sharding featurised-example under run-inference.py.

```
        def run_inference(
      self, featurised_example: features.BatchDict, rng_key: jnp.ndarray
  ) -> model.ModelResult:

    from jax.sharding import PartitionSpec as P, NamedSharding
    mesh = jax.make_mesh((2,), ('x'))
    featurised_example = utils.remove_invalidly_typed_feats(featurised_example)
    for k, v in featurised_example.items():
        if (v.size < 2):
            continue
        try:
            new_v = jax.device_put(jnp.asarray(v), NamedSharding(mesh, P('x')))
```

```
        except Exception as e:
            import pdb; pdb.set_trace()
        featurised_example[k] = new_v
        del v

    result = self._model(rng_key, featurised_example)
    result = jax.tree.map(np.asarray, result)
    result = jax.tree.map(
        lambda x: x.astype(jnp.float32) if x.dtype == jnp.bfloat16 else x,
        result,
    )
    result = dict(result)
    identifier = self.model_params['__meta__']['__identifier__'].tobytes()
    result['__identifier__'] = identifier
    return result
        ))
```

6.4. **Implementation and Optimization of CPU.**

The source code of AlphaFold3 is implemented in JAX, a Python-based library specifically designed for high-performance numerical computing and optimized array-oriented machine learning. In this implementation, JAX serves as the front end of the machine learning framework, while XLA acts as the backend, responsible for low-level optimizations and hardware-specific computations. JAX incorporates Just-In-Time (JIT) compilation via the XLA compiler, alongside robust support for automatic differentiation, which is critical for accelerating computational tasks and enabling gradient-based optimization processes. However, while the JIT functionality significantly enhances execution efficiency, it also introduces substantial challenges in debugging and complicates the achievement of seamless parallelism, particularly when targeting diverse hardware architectures.

6.4.1. *Migration from GPU to CPU.*

In the source code released by Google DeepMind, executing AlphaFold3 inference on a CPU is not supported. Specifically, the implementation does not provide an option for running inference on CPUs and includes several compute capability checks to prevent devices with insufficient computational power from executing the inference process. However, the XLA compiler, which serves as the backend of JAX, is capable of generating machine instructions for various architectures, including x86 CPUs. This discrepancy highlights the need for adaptation when migrating AlphaFold3's execution environment from GPU to CPU. Consequently, the migration process involves two primary steps:

(1) Bypassing Compute Capability Checks

Here are the codes performs compute capability, after inspection of these codes, we removed codes about checking compute capability and won't have an influence on model inference.

```
# run_alphafold.py
if _RUN_INFERENCE.value:
# Fail early on incompatible devices,
```

```
# but only if we're running inference.
gpu_devices = jax.local_devices(backend='gpu')
if gpu_devices:
  compute_capability = float(
      gpu_devices[_GPU_DEVICE.value].compute_capability
  )
  if compute_capability < 6.0:
    raise ValueError(
        'AlphaFold 3 requires at least GPU compute capability 6.0'
        '(see https://developer.nvidia.com/cuda-gpus).'
    )
  elif 7.0 <= compute_capability < 8.0:
    xla_flags = os.environ.get('XLA_FLAGS')
    required_flag = \textbf{}'--xla_disable_hlo_passes=custom-kernel-fusion-rewriter'
    if not xla_flags or required_flag not in xla_flags:
      raise ValueError(
          'For devices with GPU compute capability 7.x (see'
          ' https://developer.nvidia.com/cuda-gpus) the ENV XLA_FLAGS must'
          f' include "{required_flag}".'
      )
    if _FLASH_ATTENTION_IMPLEMENTATION.value != 'xla':
      raise ValueError(
          'For devices with GPU compute capability 7.x (see'
          ' https://developer.nvidia.com/cuda-gpus) the'
          ' --flash_attention_implementation must be set to "xla".'
      )
```

(2) Enable XLA compilation on CPU

After reading the source codes of JAX, we discover the device platform assignment mechanism. We can set system environment vaiable by `export JAX_PLATFORMS='cpu'` or add a line `jax.config.update("jax_platform_name", "cpu")` in *run_alphafold.py* before any calculation handled by JAX.

```
# jax-jax-v0.4.34/jax/_src/config.py
jax_platforms = optional_string_state(
    name='jax_platforms',
    default=None,
    help=(
        'Comma-separated list of platform names specifying which platforms jax '
        'should initialize. If any of the platforms in this list are not successfully '
        'initialized, an exception will be raised and the program will be aborted. '
        'The first platform in the list will be the default platform. '
        'For example, config.jax_platforms=cpu,tpu means that CPU and TPU backends '
        'will be initialized, and the CPU backend will be used unless otherwise '
        'specified. If TPU initialization fails, it will raise an exception. '
        'By default, jax will try to initialize all available '
```

```
        'platforms and will default to GPU or TPU if available, and fallback to CPU '
        'otherwise.'
        ))
```

(3) Ensure the compatibility of data precision on different platform

Generally, if the hardware architecture is different, the data accuracy may be different. And we do meet calculation problem which may caused by precision in Figure 19.

```
  File "/app/alphafold-cpu/run_alphafold.py", line 460, in predict_structure
    inference_results = model_runner.extract_structures(
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/app/alphafold-cpu/run_alphafold.py", line 384, in extract_structures
    return list(
           ^^^^^
  File "/alphafold3_venv/lib/python3.11/site-packages/alphafold3/model/model.py", line 375, in ge
t_inference_result
    pred_structure = get_predicted_structure(result=result, batch=batch)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/alphafold3_venv/lib/python3.11/site-packages/alphafold3/model/model.py", line 129, in ge
t_predicted_structure
    pred_struc = pred_struc.copy_and_update_atoms(
                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/alphafold3_venv/lib/python3.11/site-packages/alphafold3/structure/structure.py", line 13
55, in copy_and_update_atoms
    new_atoms = structure_tables.Atoms(
                ^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 13, in __init__
  File "/alphafold3_venv/lib/python3.11/site-packages/alphafold3/structure/structure_tables.py",
line 103, in __post_init__
    raise ValueError(
ValueError: Column x must not contain NaN/inf values.
```

Figure 19. *Met Nan/inf values problem in inference*

After comparing intermediate results of the CPU version with the GPU version in Figure 20, we can tell that, after data being processed by the diffusion module, the results turned into nan as shown in Figure 21.

Then we use the Python Debugger to locate the root of the nan problem, finally we locate the problem in *src/alphafold3/model/network/diffusion_head.py*, partial codes are as follows:

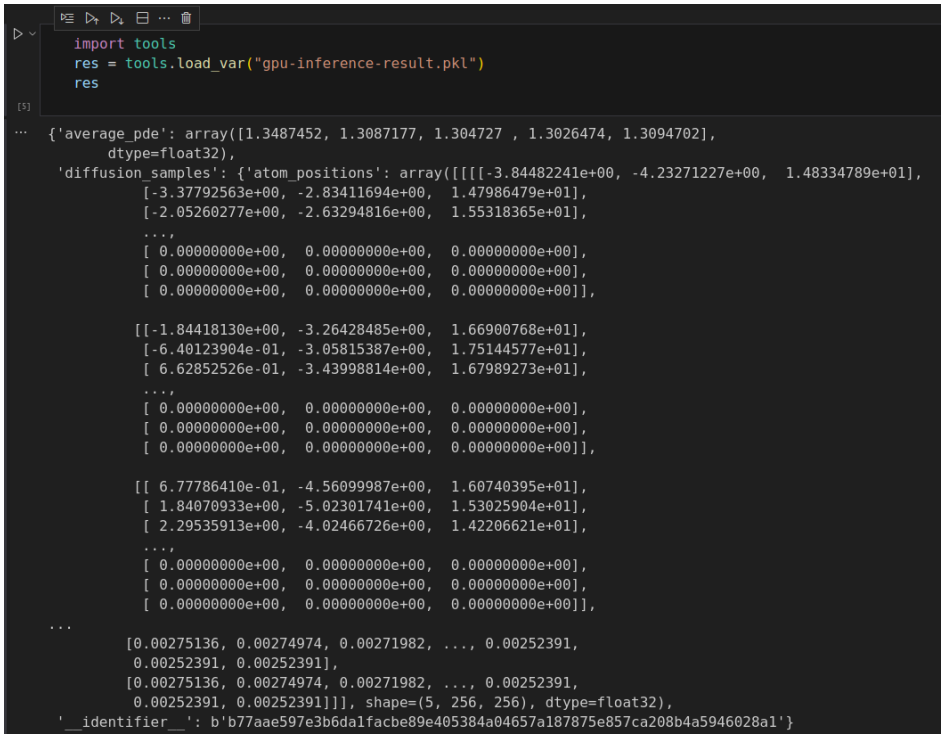
```
# src/alphafold3/model/network/diffusion_head.py
# fix CPU inference NaN problem
tmp = t_hat**2 - noise_level_prev**2
tmp = jnp.where(jnp.abs(tmp) < 1e-6, 0.0, tmp)
noise_scale = config.noise_scale * jnp.sqrt(tmp)

# original code
# noise_scale = config.noise_scale * jnp.sqrt(t_hat**2 - noise_level_prev**2)
```

Due to the equation *t_hat**2 - noise_level_prev**2* lack of positive or negative judgement, it will generate negative values which violate the rule of absolute value. Thus we change the code to handle negative value correctly.

6.4.2. *XLA optimization.*

The JAX-XLA collaboration process consists of a JAX frontend that defines abstract data structures and network structures to generate computation graphs and transform them into HLOs, a form



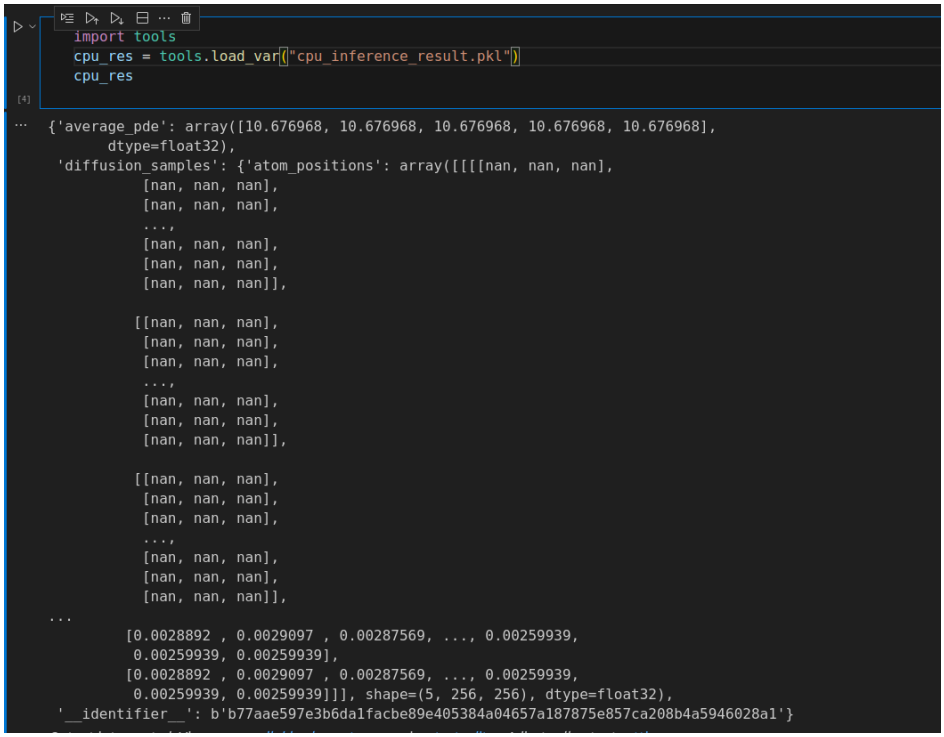
```
import tools
res = tools.load_var("gpu-inference-result.pkl")
res
```

```
[5]
{'average_pde': array([1.3487452, 1.3087177, 1.304727 , 1.3026474, 1.3094702],
      dtype=float32),
 'diffusion_samples': {'atom_positions': array([[[[-3.84482241e+00, -4.23271227e+00,  1.48334789e+01],
          [-3.37792563e+00, -2.83411694e+00,  1.47986479e+01],
          [-2.05260277e+00, -2.63294816e+00,  1.55318365e+01],
          ...,
          [ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00],
          [ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00],
          [ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00]],

         [[-1.84418130e+00, -3.26428485e+00,  1.66900768e+01],
          [-6.40123904e-01, -3.05815387e+00,  1.75144577e+01],
          [ 6.62852526e-01, -3.43998814e+00,  1.67989273e+01],
          ...,
          [ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00],
          [ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00],
          [ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00]],

         [[ 6.77786410e-01, -4.56099987e+00,  1.60740395e+01],
          [ 1.84070933e+00, -5.02301741e+00,  1.53025904e+01],
          [ 2.29535913e+00, -4.02466726e+00,  1.42206621e+01],
          ...,
          [ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00],
          [ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00],
          [ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00]],
...
        [0.00275136, 0.00274974, 0.00271982, ..., 0.00252391,
         0.00252391, 0.00252391],
        [0.00275136, 0.00274974, 0.00271982, ..., 0.00252391,
         0.00252391, 0.00252391]]], shape=(5, 256, 256), dtype=float32),
 '__identifier__': b'b77aae597e3b6da1facbe89e405384a04657a187875e857ca208b4a5946028a1'}
```

Figure 20. *GPU version inference intermediate result*



```
import tools
cpu_res = tools.load_var("cpu_inference_result.pkl")
cpu_res
```

```
[4]
{'average_pde': array([10.676968, 10.676968, 10.676968, 10.676968, 10.676968],
      dtype=float32),
 'diffusion_samples': {'atom_positions': array([[[[nan, nan, nan],
          [nan, nan, nan],
          [nan, nan, nan],
          ...,
          [nan, nan, nan],
          [nan, nan, nan],
          [nan, nan, nan]],

         [[nan, nan, nan],
          [nan, nan, nan],
          [nan, nan, nan],
          ...,
          [nan, nan, nan],
          [nan, nan, nan],
          [nan, nan, nan]],

         [[nan, nan, nan],
          [nan, nan, nan],
          [nan, nan, nan],
          ...,
          [nan, nan, nan],
          [nan, nan, nan],
          [nan, nan, nan]],
...
        [0.0028892 , 0.0029097 , 0.00287569, ..., 0.00259939,
         0.00259939, 0.00259939],
        [0.0028892 , 0.0029097 , 0.00287569, ..., 0.00259939,
         0.00259939, 0.00259939]]], shape=(5, 256, 256), dtype=float32),
 '__identifier__': b'b77aae597e3b6da1facbe89e405384a04657a187875e857ca208b4a5946028a1'}
```

Figure 21. *CPU version inference intermediate result*

of XLA's intermediate representation. The XLA backend is responsible for compiling and executing these graphs. Based on Linux system performance monitoring tools htop monitoring results, when using CPU for inference, most core occupancy rate is quite low(Figure 22).

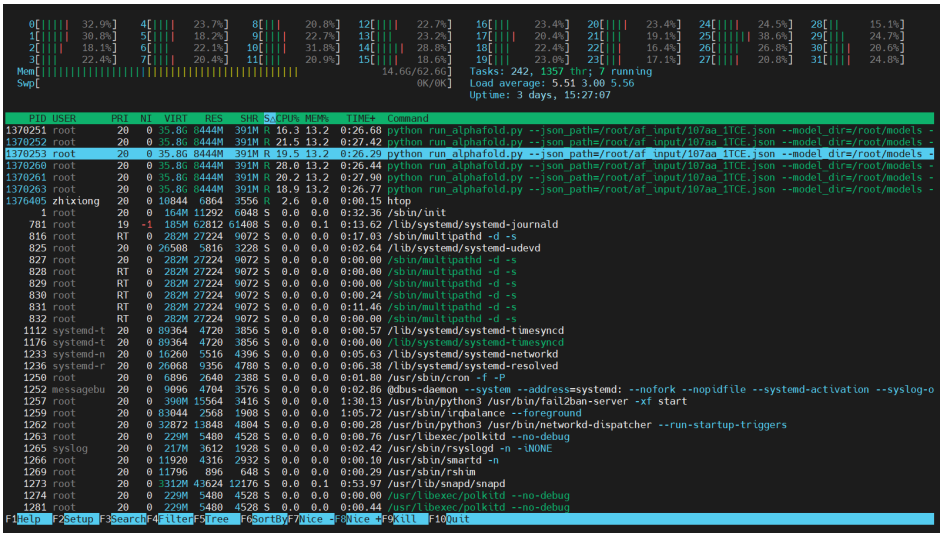


Figure 22. *Low CPU occupancy rate*

By contrast, the GPU's occupancy rate is always close to 100%(Figure 18), while the CPU occupancy rate is always low. We hypothesize that all the network and data are well structured by JAX frontend, while XLA backend affects the overall performance. Thus we made an effort to optimize the performance of XLA backend. Our methods are as follows:

(1) **Build `jaxlib` from source**

It means compile `jaxlib` from source using the LLVM compiler. `Jaxlib` is a library that provides C++ support for JAX frontend and includes xla_extension. So that provides XLA backend calls, meaning that we can use the native LLVM compiler with appropriate compilation options to improve XLA backend performance.

Optional, the manufacturer of our CPU, AMD, offers AMD Optimizing C/C++ and Fortran Compilers (AOCC) and AMD Optimizing CPU Libraries (AOCL). The AOCC and AOCL supports a lot of AMD "Zen" core architecture specified optimizations which may improve the performance of the machine codes generated by XLA.

Here are our testing results with different `jaxlib`:

First of all, we build `jaxlib` with native LLVM 18 comparing with original `jaxlib` provided by Google DeepMind. Our LLVM 18 set the compile option `march='native'` And the result tells that the `jaxlib` built with native LLVM 18 is slower than the original `jaxlib` provided by Google DeepMind. We assume that maybe the Google DeepMind's `jaxlib` enable MKL/BLAS support.

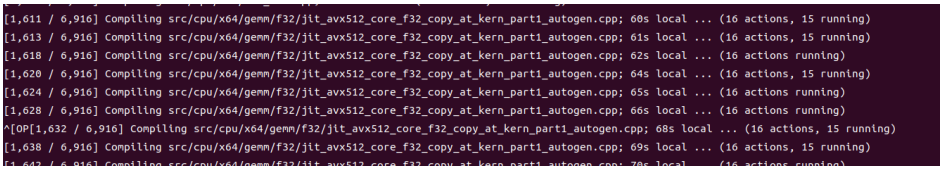


Figure 23. *Build jaxlib from source using bazel*

AOCL optimization

In the recompilation environment, we keep the other parameters unchanged, and compare the running time of the parameter result "rebuild aocl" which means rebuilding jaxlib with

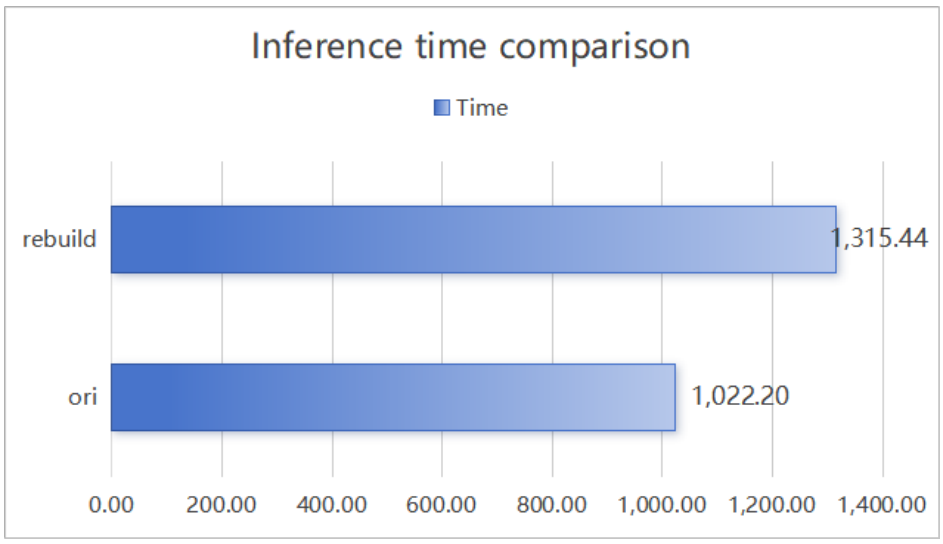


Figure 24. *Compare inference time between Google's original jaxlib and our rebuild jaxlib*

LLVM 18 and linking AOCL. And the result "rebuild" that are not linking AOCL, and we conclude that the operation time of the program is partially improved after using the AOCL library as depicted in Figure 25.

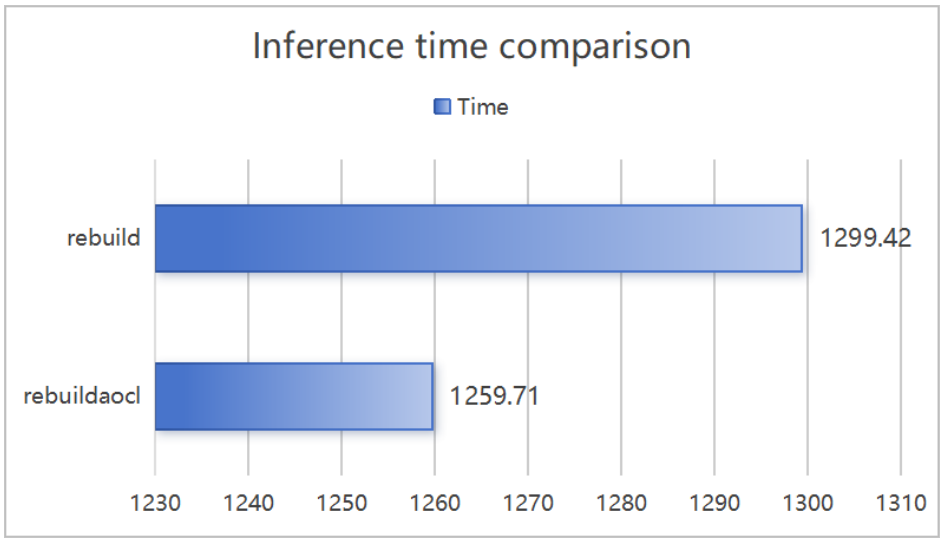


Figure 25

Then we tried to use AOCC, but AOCC is based on LLVM 17, which isn't support build `jaxlib` from source as `jaxlib` requires LLVM 18 or later version.

(2) **AVX512_BF16 Instruction set optimization**

Enabling AVX512_BF16 can significantly enhance the inference speed by making the most of vector registers for SPMD tasks performance improvement and leveraging hardware acceleration for bfloat16 (BF16) data types. BF16 is a 16-bit floating-point format that offers a balance between computational efficiency and numerical stability, making it particularly well-suited for deep learning workloads. After checking CPU occupancy rate

(Figure 22), we are doubt whether the AVX512_BF16 instruction set is being adopted. Thus we read the source codes of XLA to ensure AVX512_BF16 instruction set is set correctly.

To activate XLA support for AVX512_BF16, we need to run

```
export XLA_FLAGS="--xla_cpu_max_isa=AVX512_BF16"
```

```
flag_list->push_back(tsl::Flag(
    "xla_cpu_max_isa",
    uppercase_string_setter_for(&DebugOptions::set_xla_cpu_max_isa),
    debug_options->xla_cpu_max_isa(),
    "Maximum ISA that XLA:CPU LLVM backend will codegen, i.e., it will not "
    "use newer instructions. Available values: SSE4_2, AVX, AVX2, AVX512, "
    "AVX512_VNNI, AVX512_BF16, AMX, and AMX_FP16. (`AMX` will enable both "
    "`AMX_BF16` and `AMX_INT8` instructions.)"));

```

Then we compared the inference time between inference with AVX512_BF16 and inference without AVX512_BF16.

The AVX512_BF16 instruction set is used to accelerate program execution, under the condition that other factors are controlled. A comparison of the execution times whether enabling AVX512_BF16 or not, the result reveals that AVX512_BF16 outperforms no AVX512_BF16, demonstrating a shorter runtime, as we measured, the inference time reduced for 5%.

And according to AVX512_BF16 instruction set design concept, the vector register can store data up to 512 bits. Thus, to improve the inference performance, we can optimize the LLVM preferred vector width to 512 bits to make the most of the vector register by adding `--xla_cpu_prefer_vector_width=512` in XLA_FLAGS.

(3) **Parallel strategy optimization**

We perform a comparison between whether define vector_width size to 512 bits.

LLVM preferred vector size optimization

Vector 512 is the vector size generated by LLVM after setting the XLA backend, setting it to 512 means allowing LLVM to generate a 512-bit vector as much as possible, so that the vector register can be fully utilized, under the condition that other factors are controlled. Under the AVX512_BF16 instruction set optimization, we performed a comparison of the inference runtime about whether enable preferred vector size to 512 or not. And the results demonstrates that vector512 achieves better performance with a shorter runtime, as we measured, the inference time reduced over 1%.

After all the previous XLA optimization attempts, our CPU occupation is still low. We cannot help wondering whether JAX's Just-In-Time (JIT) compilation functionality would handle parallel control on the CPU properly, thus we disable the JIT functionality to verify our hypothesis.

JIT setting optimization

JIT optimization in the JAX framework may not efficiently handle multi-core and multi-device scenarios. Therefore, we ensure that other factors are the same when turning off JIT and turning on JIT. We performed another comparison between JIT enabled and JIT disabled. The result reveals that `JIT disabled` outperforms `JIT enabled`, exhibiting a shorter runtime, as illustrated in Figure 26.

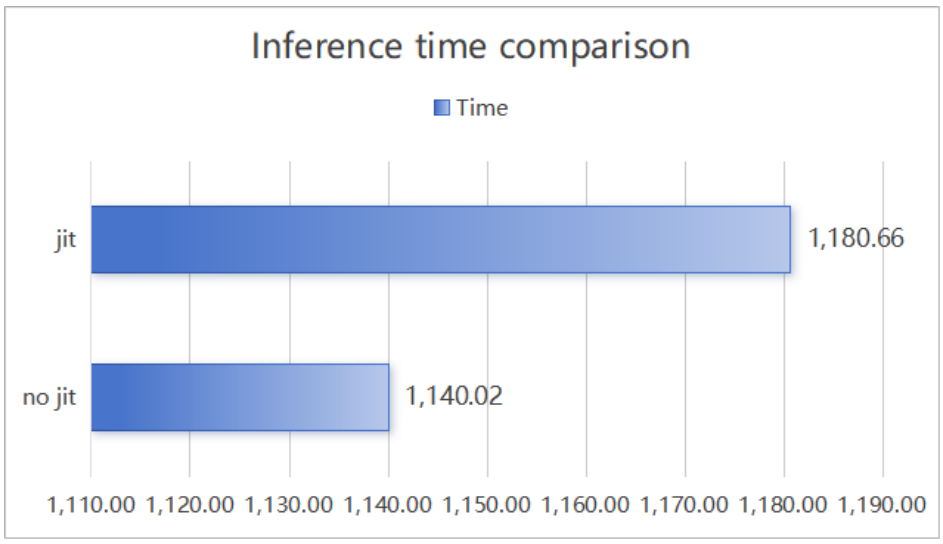


Figure 26

After gain performance improvement by disabling JIT, we tried to control the parallel mechanism manully. Our CPU is with 16 cores. Thus we tried make the most of these cores with different parallel device allocation mechanisum for JAX and XLA. XLA's multi-thread mechanisum is handled by Eigen's threadpool, a threadpool implementation by high efficiency linear algebra library Eigen. Whereas, we cannot determine whether the original `jaxlib` compilation using MKL or BLAS, which may influence the parallel mechanism in many ways. Also, we discover a number of times calling `futex` kernel system call when running inference (Figure 27).

Thus, we attempted to customize different parallel configurations to figure out the better parallel strategy.

Trying to make JAX handle all the devices

In JAX framework, a single CPU core can be mapped to a single divice. Thus we changed jax_devices to 16, allowing JAX to manage all the 16 CPU cores, under the condition that other factors are controlled. We performed a comparison of the execution runtime between JAX handling all the CPU cores demonstrates that `_16dev` outperforms the configuration without `_16dev`, achieving a shorter runtime, as illustrated in Figure 28.

Comparison between JAX handled parallel and XLA backend handled parallel

For the reason of complex parallel mechanism mixed with JAX and XLA, we tried to find out the better configuration to implement better parallelism. Thus we conducted a comparison between JAX handled parallel and XLA backend handled parallel. The "16 dev" means JAX handled all the 16 CPU cores, the "1 omp" means XLA backend parallel is strictly restricted. In a similar way for "16 omp" and "1 dev". And we also tested on our rebuild jaxlib. As is shown in Figure 29 and Figure 30, the JAX handled parallel got better efficiency in parallel. Both on the Google DeepMind jaxlib and our rebuild jaxlib.

6.4.3. *Operator optimization for Einstein Summation Convention.*

In the field of mathematical physics, especially in the research involving tensor analysis and general relativity, the Einstein summation convention, as an efficient tensor operation representation, is widely used in the description of coordinate transformations in the fields of relativistic field

% time	seconds	usecs/call	calls	errors	syscall
97.20	76.239750	17303	4406	99	futex
1.39	1.092248	18	58311		munmap
0.97	0.759616	12	59334		mmap
0.29	0.226046	20	10885		read
0.05	0.036033	9	3890		gettid
0.04	0.035143	7	4751		brk
0.02	0.014826	1	10297	970	newfstatat
0.01	0.009454	2	4128		sched_getaffinity
0.01	0.006662	0	10460	1	lseek
0.00	0.003662	130	28		write
0.00	0.002843	23	121		clone
0.00	0.002391	1	2289	341	openat
0.00	0.001950	4	458		mprotect
0.00	0.000925	0	1949		close
0.00	0.000884	1	678		getdents64
0.00	0.000388	0	1364	1361	ioctl
0.00	0.000318	2	121	121	clone3
0.00	0.000302	1	245		rt_sigprocmask
0.00	0.000103	103	1		execve
0.00	0.000052	0	141		pread64
0.00	0.000016	2	8	4	readlink
0.00	0.000009	1	9		getrandom
0.00	0.000008	0	77		rt_sigaction

Figure 27. *Frequent futex call when running inference*

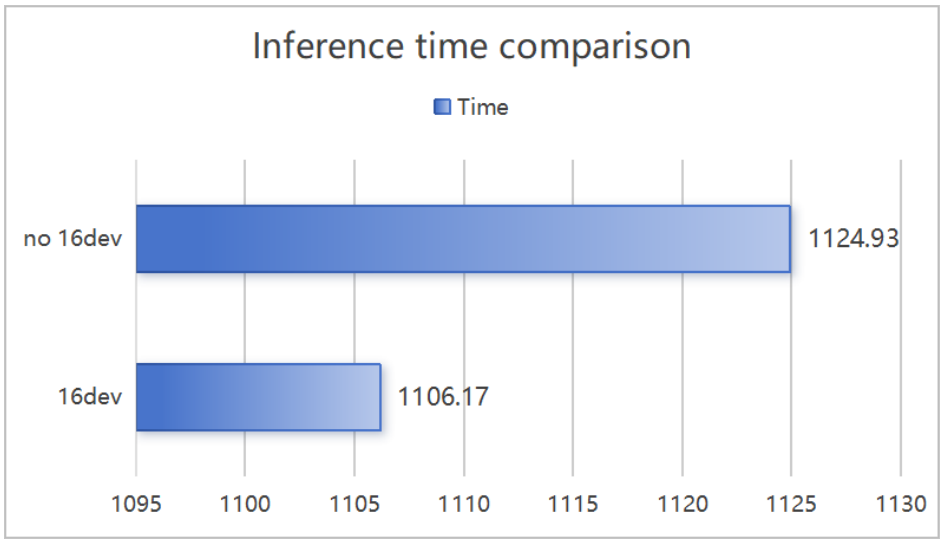


Figure 28

equations and continuum mechanics. This convention significantly simplifies tensor expressions with multiple summation operations through a streamlined indicator labeling system.

According to this convention, the summation of all possible values of the indicator is performed by default if and only if there are paired superscript and subscript Latin letter indicators in the same term. The range of values of the index usually depends on the spatial properties studied: 1, 2, 3 in 3D Euclidean space, and 0, 1, 2, 3 in 4D Minkowski spacetime or general space-time manifolds. It should be noted that although the index value range of scalar tensors (zero-order tensors) can

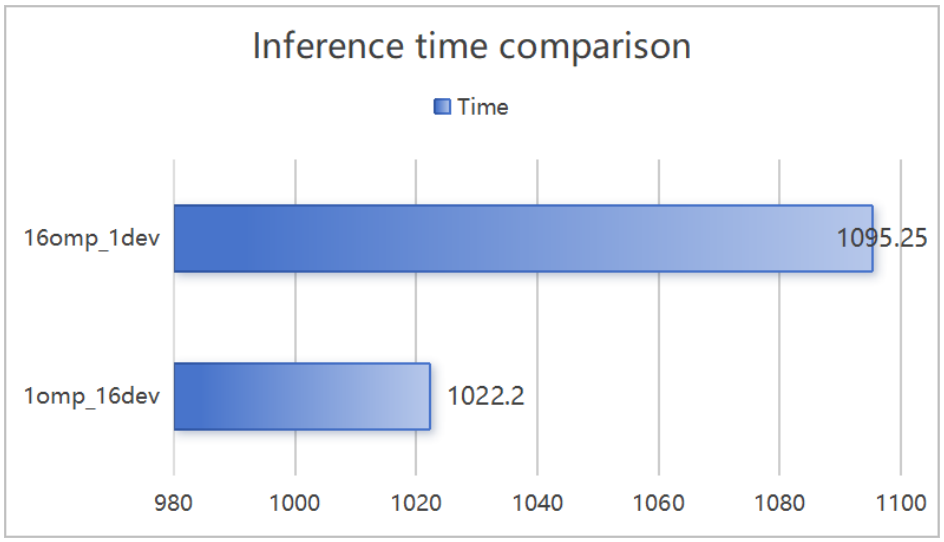


Figure 29

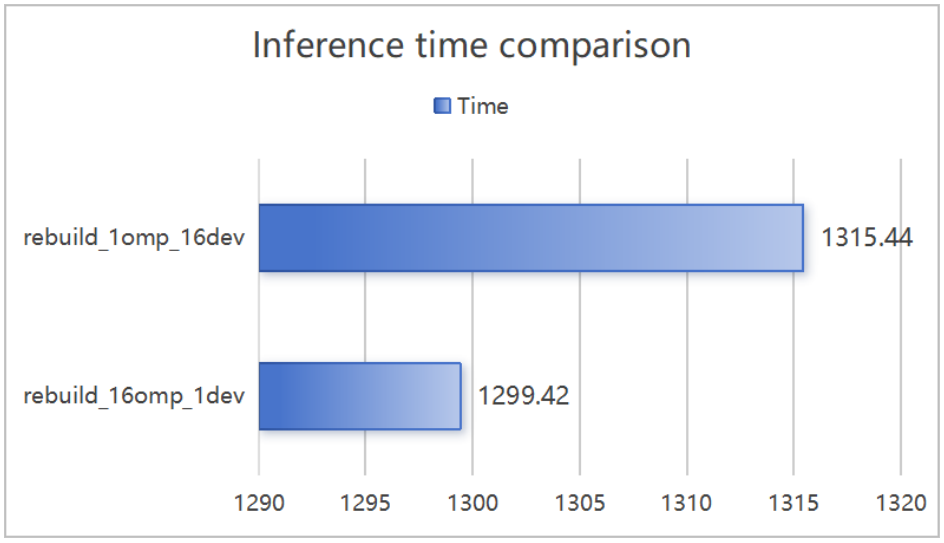


Figure 30

theoretically be extended to any cardinal set, they are usually limited to countable sets in physical application scenarios, and need to be supported by rigorous mathematical arguments in the case of infinite-dimensional spaces.

In this way, in n-dimensional space,

$$y = c_1 x^1 + c_2 x^2 + c_3 x^3 + \cdots + c_n x^n \tag{6.1}$$

This is usually written as a summation formula:

$$y = \sum_{i=1}^{n} c_i x^i \tag{6.2}$$

and simplified to

$$y = c_i x^i \tag{6.3}$$

Under a basis transformation, a scalar remains invariant. When the basis changes, the linear transformation of a vector is described by a matrix, whereas that of a covector is depicted by the inverse of this matrix. This design ensures that the linear function associated with the covector remains unchanged regardless of the basis transformation. Given that only the sum is constant and each component involved in the summation can vary, Einstein introduced a convention where repeating an index implies summation over that index.

To identify the most efficient contraction path, one can exhaustively explore all possible ways of combining tensors, including those involving newly formed intermediate tensors. Although this exhaustive search algorithm scales factorially with the number of tensors (i.e., $O(N!)$), making it potentially more computationally expensive than the unoptimized contraction itself, it serves as an invaluable benchmark for evaluating alternative strategies. In the opt_einsum library, this optimal path is determined by the function opt_einsum.paths. optimal, which employs a recursive, depth-first search approach. By maintaining a record of the best path found so far—measured in terms of the total estimated floating-point operations (FLOPs)—the algorithm efficiently prunes suboptimal paths as soon as their FLOP count exceeds the current best. This ensures that the search remains tractable for small to moderately sized tensor networks. By default, the optimize='auto' mode in opt_einsum utilizes this optimal strategy for expressions involving up to four tensors. However, even for larger networks comprising 9–10 tensors, the algorithm can compute the optimal path within seconds, making it a practical choice for many applications.

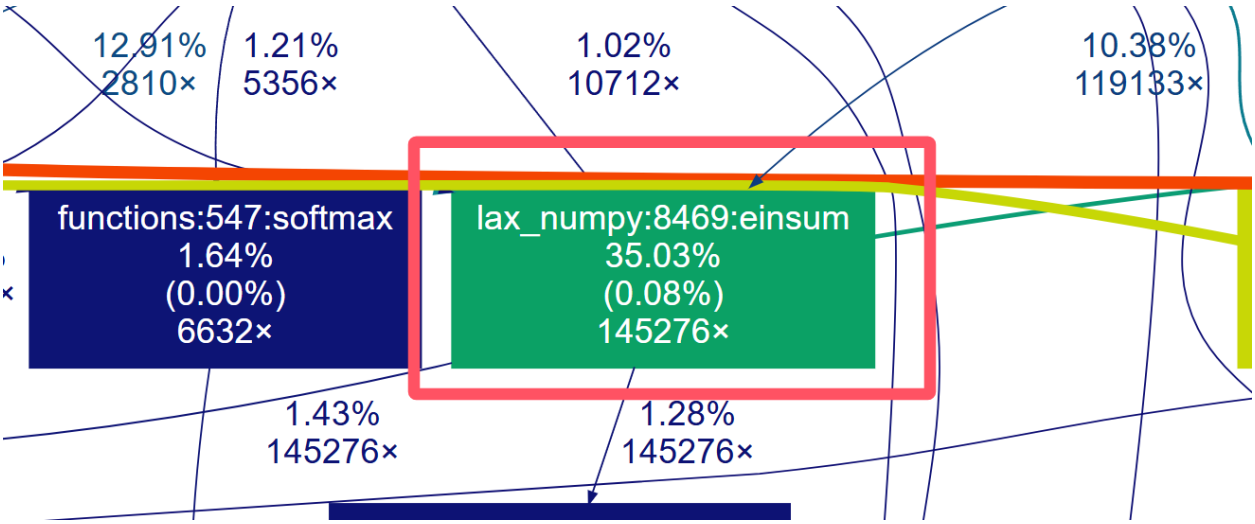


Figure 31. *A high-cost function in function call graph*

Through the above function call graph, we locate that the computation time of the einsum operator accounts for more than one-third of the total program running time, so our subsequent approach is mainly to optimize the einsum operation to maximize the reduction in running time.

(1) **Optimization about enabling optimized einsum in attention module**
As we found the einsum function runtime occupied $\frac{1}{3}$ inference runtime, we tried to use optimized einsum. The comparison result is shown in Figure 32.

(2) **Comparison between use greedy optimization in einsum**

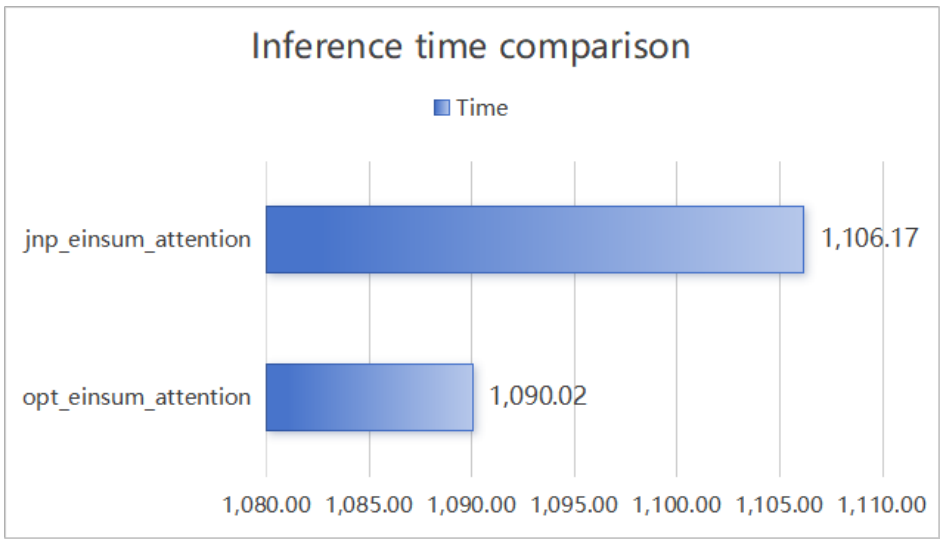


Figure 32

After the optimized einsum improved the performance of einsum, we tried to lead in greedy optimization in the solving process of einsum. Whereas the greedy optimization didn't work, even worse than the default solving process of einsum. The result is as follows:

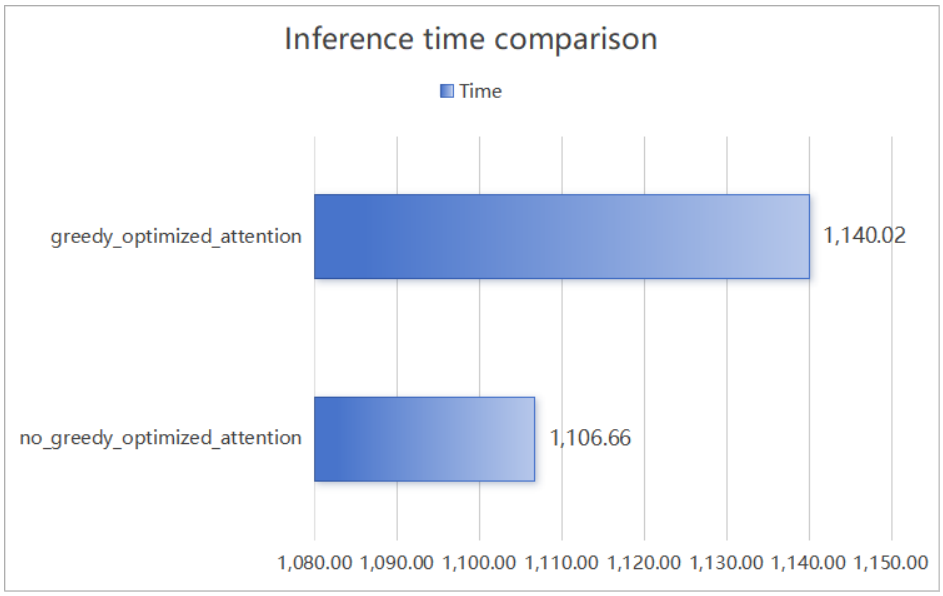


Figure 33

6.5. **Experiments.**

6.5.1. *Setup.*

Hardware

Unless otherwise stated, the system is powered by the AMD EPYC 9124 processor, 128GB of DDR5 RAM and dual NVIDIA GeForce 4070 Ti Super GPUs.

Software

Ubuntu version 22.04 LTS, CUDA version 12.4, Python version 3.11.11, dm-haiku version 0.0.13, jax version 0.4.34, `jaxlib` version 0.4.34, opt_einsum version 3.4.0, triton version 3.1.0.

Datasets

In this competition, the dataset provided by the official organizers of the competition was adopted. This dataset is presented in the form of JSON files, with a total of 12 JSON files included.

Comparison methods

To evaluate the effectiveness of our optimization strategies, we conducted a comparative analysis between the baseline implementation and the optimized implementation. The comparison was performed under the following framework:

(1) **Baseline Implementation:** The baseline version refers to the execution result from executing the original AlphaFold3 inference code provided by Google DeepMind at Github.
(2) **Optimized Implementation:** Our optimized implementation focuses on reducing inference time while ensuring that the prediction quality remains consistent with the baseline.

6.5.2. *GPU Evaluation Result.*

Inference time

In terms of runtime, we compared the Dual version with the Baseline version, as shown in Table 11. **The results show that the dual-GPU-optimized version should outperform the baseline implementation, but because we did not solve the communication problem between the GPUs, the communication overhead is too large, and the data presented appears to be less effective.**

Table 11. *Comparison of running times between the baseline and the optimized version on dual GPUs*

Input \ Index	Baseline	Dual GPUs
37aa-2jo9	43.35 s	56.14s
107aa-1tce	43.52 s	56.15s
301aa-3db6	68.43 s	98.63s
436aa-1muu	68.60 s	98.76s
583aa-1cf3	110.20 s	181.47s
740aa-4a5s	110.70 s	181.59s
959aa-7lzh	158.03 s	309.43s
1024aa-5l6h	158.46 s	308.44s
1284aa-4xwk	309.92 s	734.89s
1290aa-1g9a	309.50 s	731.64s
1303aa-1t3t	309.95 s	730.10s
1491aa-5kis	311.82 s	730.84s

Accuracy

In terms of accuracy, we compared the `summary_confidences.json` files from the output of our optimized version with those of the Baseline version, as shown in Table 12. **The results indicate that our optimized version achieves higher prediction quality.** The predicted structures are very close to the true structures, demonstrating overall high prediction accuracy. Furthermore, the predicted structures are geometrically reasonable, ensuring both precision and validity in the predictions.

Table 12. *Accuracy comparison across different inputs (Data left is baseline and right is Dual-GPU version).*

Index / Input	chain-pair-iptm	chain-pair-pae-min	chain-ptm	has-clash	ptm
37aa-2jo9	0.69 / 0.68	0.76 / 0.76	0.69 /0.68	0.0 / 0.0	0.69 /0.68
107aa-1tce	0.82 / 0.82	0.76 / 0.76	0.82 / 0.82	0.0 / 0.0	0.82 / 0.82
301aa-3db6	0.93 / 0.93	0.76 / 0.76	0.93 / 0.93	0.0 / 0.0	0.93 / 0.93
436aa-1muu	0.91 / 0.91	0.76 / 0.76	0.91 / 0.91	0.0 / 0.0	0.91 / 0.91
583aa-1cf3	0.98 / 0.98	0.76 / 0.76	0.98 / 0.98	0.0 / 0.0	0.98 / 0.98
740aa-4a5s	0.96 / 0.96	0.76 / 0.76	0.96 / 0.96	0.0 / 0.0	0.96 / 0.96
959aa-7lzh	0.66 / 0.66	0.76 / 0.76	0.66 / 0.66	0.0 / 0.0	0.66 / 0.66
1024aa-5l6h	0.96 / 0.96	0.76 / 0.76	0.96 / 0.96	0.0 / 0.0	0.96 / 0.96
1284aa-4xwk	0.79 / 0.79	0.76 / 0.76	0.79 / 0.79	0.0 / 0.0	0.79 / 0.79
1290aa-1g9a	0.74 / 0.74	0.76 / 0.76	0.74 / 0.74	0.0 / 0.0	0.74 / 0.74
1303aa-1t3t	0.98 / 0.98	0.76 / 0.76	0.98 / 0.98	0.0 / 0.0	0.98 / 0.98
1491aa-5kis	0.95 / 0.95	0.76 / 0.76	0.95 / 0.95	0.0 / 0.0	0.95 / 0.95

6.5.3. *CPU Evaluation Result.*

Due to lack of time and computing resources, baseline version of CPU inference results was performed on the **cloud server** which was powered by AMD EPYC 7R13 with 48 cores, 384 GB RAM, but we only enable 16 cores with virtualization technology for the purpose of making **the cloud server hardware topologies more similar with our local server**. If we use the results from the **cloud server**, we will add **"(cloud server)"** identifier next to the data. As for the inference results performed on our local server, it won't have special identifier. Our server was powered by AMD EPYC 9124 with 16 cores, 128 GB RAM.

All the "Optimization" result in this evaluation, we chosen the setting with best inference performance in Implementation and Optimization of CPU in Section 6.4, the details are as follows:

(1) Disable JAX's JIT
(2) Let JAX handle devices rather than XLA
(3) Enable AVX512_BF16 support for XLA and adjust vector size to 512 to better match the vector registers' size

Inference time

Table 13 illustrates baseline version inference time running of 12 data sets on the cloud server. It is evident that the trends in execution durations for the different CPU versions align with those of the GPU versions. However, there is a significant disparity in execution time, ranging from approximately 20 to 70 times. Given the performance gap in floating-point peak performance between GPUs and CPUs in HPL and HPCG (where GPU performance is roughly equivalent to that of CPU), it can be inferred that there remains substantial room for optimization in the CPU versions, potentially by a factor of 11.61 times as shown in equation 6.5 The FP16 theoretical performance of our GPU NVIDIA 4070 Ti Super is 44.10 TFlops and the FP64 theoretical performance of our CPU is 947.2 GFlops. To transform the FP64 performance value into FP16, we need to multiply FP64 performance value by 4. Thus our CPU FP16 theroretical performance is

$$\text{CPU FP16 theroretical performance} = 947.2\text{GFlops} * 4 = 3788.8\text{GFlops} = 3.8\text{TFlops} \quad (6.4)$$

Table 13. *CPU inference baseline results across different inputs on the cloud CPU server*

Input \ Index	Baseline
37aa-2jo9	1014.81 s
107aa-1tce	1011.02 s
301aa-3db6	3730.45 s
436aa-1muu	3724.15 s
583aa-1cf3	8600.06 s
740aa-4a5s	8594.12 s
959aa-7lzh	18719.31 s
1024aa-5l6h	18891.80 s
1284aa-4xwk	49184.80 s
1290aa-1g9a	49360.46 s
1303aa-1t3t	49223.42 s
1491aa-5kis	49172.02 s

Then we can calculate the theroretical performance ratio between GPU and CPU.

$$\text{Performance Ratio} = \frac{\text{GPU FP16 performance}}{\text{CPU FP16 performance}} = \frac{44.10\text{TFlops}}{3.80\text{TFlops}} = 11.61 \tag{6.5}$$

Table 14 presents the execution times of the representative 5 data sets before and after optimization on our local CPU server. It can be observed that due to the smaller memory capacity of the local CPU server, its peak performance is inferior to that of the cloud server. Therefore, **the execution time of the unoptimized version of the local CPU server is slightly longer than that of the cloud server, but the difference is on the order of magnitude. The optimized version can achieve an acceleration of 1.12 to 1.16 times, with an average of 1.164 times.**

Table 14. *Comparison about CPU inference time between the baseline and the optimized version on our local CPU server*

Input \ Index	Baseline	Optimization	Time Saved	**Speedup**
37aa-2jo9	1315.50 s	1176.71 s	138.79 s	1.12×
107aa-1tce	1269.16 s	1024.64 s	244.52 s	1.24×
301aa-3db6	3393.60 s	2957.68 s	435.92 s	1.15×
436aa-1muu	3413.45 s	2955.84 s	457.61 s	1.15×
1024aa-5l6h	11994.38 s	10300.25 s	1694.13 s	1.16×

Accuracy The results of the accuracy test are shown in Table 15, where the differences in accuracy between the CPU version and the GPU are indicated by the values in brackets in the table. **Therefore, the CPU version meets the accuracy requirements of the competition.**

6.6. **Optimization Plans.**

6.6.1. *AlphaFold 3 inference on GPU.*

Table 15. *Baseline of CPU inference results accuracy across different inputs on the cloud server*

Input \ Index	chain-pair-iptm	chain-pair-pae-min	chain-ptm	has-clash	ptm
37aa-2jo9	0.68(0)	0.76(0)	0.68(-0.01)	0.0(0)	0.68(-0.01)
107aa-1tce	0.82(0)	0.76(0)	0.82(0)	0.0(0)	0.82(0)
301aa-3db6	0.93(0)	0.76(0)	0.93(0)	0.0(0)	0.93(0)
436aa-1muu	0.91(0)	0.76(0)	0.91(0)	0.0(0)	0.91(0)
583aa-1cf3	0.98(0)	0.76(0)	0.98(0)	0.0(0)	0.98(0)
740aa-4a5s	0.96(0)	0.76(0)	0.96(0)	0.0(0)	0.96(0)
959aa-7lzh	0.67(+0.01)	0.76(0)	0.67(+0.01)	0.0(0)	0.67(+0.01)
1024aa-5l6h	0.96(0)	0.76(0)	0.96(0)	0.0(0)	0.96(0)
1284aa-4xwk	0.79(0)	0.76(0)	0.79(0)	0.0(0)	0.79(0)
1290aa-1g9a	0.74(0)	0.76(0)	0.74(0)	0.0(0)	0.74(0)
1303aa-1t3t	0.98(0)	0.76(0)	0.98(0)	0.0(0)	0.98(0)
1491aa-5kis	0.95(0)	0.76(0)	0.95(0)	0.0(0)	0.95(0)

(1) **Plan 1:** We applied data parallelism to the Input Embedder module of AlphaFold3, enabling dual-GPU processing as shown in Figure 34. However, as indicated in the results, the performance gain was limited, potentially due to high inter-device communication overhead arising from the large data size. Future work will focus on optimizing communication efficiency to reduce inference time.
(2) **Plan 2:**We observed that the Encoder component of AlphaFold3 involves multiple iterative computations. An attempt was made to parallelize these operations; however, the effort was unsuccessful, likely due to mismatched axis sizes during subsequent alignment. Future work will focus on resolving the data dimension compatibility issues to achieve optimization.
(3) **Plan 3:**Parallelization was attempted in the computation component of the Diffusion module in AlphaFold3. However, errors occurred due to incompatibility between JAX's parallelization and the Haiku library. Future work will explore alternative strategies to achieve optimized performance.

6.6.2. *AlphaFold 3 inference on CPU.*

After analysing the flame graph of inference, we discover the Pairformer module remained to be optimized.

(1) Data Parallelism
  (a) Model Parameter Partitioning
  Based on our previous attempts to implement data parallelism in Alphafold3, we observe that during Pairformer iterations, computational data can be distributed across multiple devices along the channel dimension of the pair representation. Concurrently, the model parameters can be partitioned, enabling parallel inference across multiple devices without duplicating the model parameters. This method ensures that the model inference results remain consistent, while improving computational efficiency by leveraging data parallelism. This approach is inspired by prior work on parameter distribution strategies in large models.
  (b) Matrix Block Implementation Based on Hardware Topology

```
| NVIDIA-SMI 550.144.03             Driver Version: 550.144.03     CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 4070 ...    On |   00000000:C1:00.0 Off |                  N/A |
|  0%   33C    P0            103W /  285W |   15478MiB /  16376MiB |     100%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+
|   1  NVIDIA GeForce RTX 4070 ...    On |   00000000:C2:00.0 Off |                  N/A |
|  0%   35C    P0            101W /  285W |   15476MiB /  16376MiB |     100%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+---------------------------------------------------------------------------------------+
| Processes:                                                                            |
|  GPU   GI   CI        PID   Type   Process name                            GPU Memory |
|        ID   ID                                                             Usage      |
|=======================================================================================|
|    0   N/A  N/A   4130506      C   python                                    15468MiB |
|    1   N/A  N/A   4130506      C   python                                    15466MiB |
+---------------------------------------------------------------------------------------+
```

Figure 34. *Running status of dual GPUs*

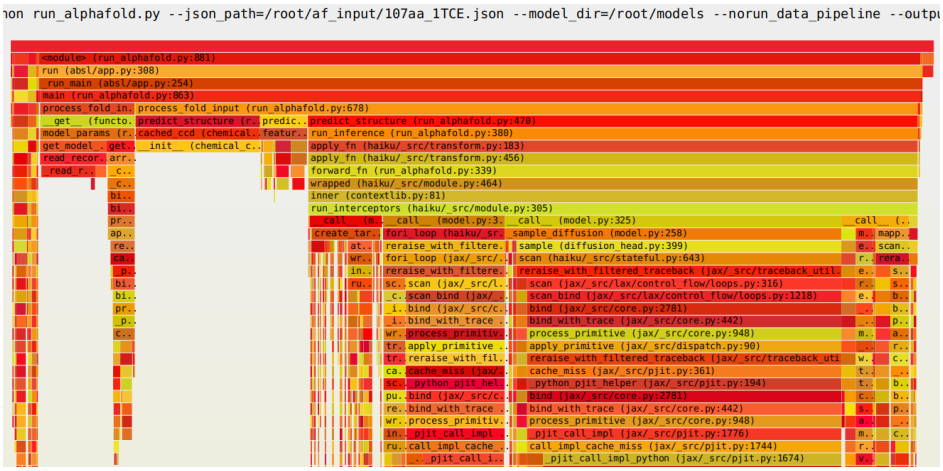


Figure 35. *Inference process flame graph on CPU*

In a multi-device environment, such as a multi-core CPU setup, matrix block operations can be effectively implemented by utilizing JAX's device mesh functionality. When a Non-Uniform Memory Access (NUMA) architecture is established across multiple devices, the performance can be further optimized by considering the improved CPU-memory access relations. This hardware-aware matrix partitioning strategy is critical for efficient parallel computation in large-scale neural networks.

(2) XLA Backend Optimization

(a) Eigen Threadpool Implementation in XLA Parallelism

The current XLA backend shows low parallel efficiency when operating on CPUs. It is necessary to investigate the implementation process of the Eigen threadpool in XLA parallelism, and how the JAX Just-In-Time (JIT) compilation mechanism influences the parallel performance. A deeper understanding of thread management within XLA, particularly in conjunction with JIT, could lead to optimizations that enhance parallel execution. Previous research has highlighted the importance of tuning thread pools and JIT mechanisms to maximize parallel efficiency.

(b) Optimizing LLVM Performance and Linking Mathematical Libraries
LLVM performance can be further improved through optimizations in the compilation and linking processes. In particular, previous attempts to use AMD Optimized CPU Libraries (AOCL) in the backend could significantly speed up the execution of complex mathematical operations, which are essential in Alphafold3's protein folding simulations. By linking high-performance libraries, such as Intel MKL or AMD AOCL, to the XLA backend, computational efficiency can be significantly improved compared to the XLA backend without an optimized library, but there still remains a gap to the Google DeepMind's XLA in the jaxlib.

6.7. **Conclusion.**

In this proposal, our main tasks include GPU and CPU inference optimization for AlphaFold3. In GPU optimization, we implemented dual GPUs for data parallelism. However, because the communication overhead is inefficient, we failed to speed up and we plan to solve it in the future. As for the CPU inference, we first utilized the JAX technique for immigration. Then we do some XLA backend and model operator optimization to accelerate the CPU inference. In the end, we achieved a 1.164 times speedup for CPU inference compared to the basic migrated CPU version. Further, we do some optimization plans based on our insight observation during the competition.

7. RNA m^5C Modification Site Detection and Performance Optimization

7.1. **Introduction.**

RNA 5-methylcytosine (RNA m^5C) is a crucial epigenetic modification in which a methyl group is added to the 5-carbon of cytosine, regulating gene expression by affecting RNA stability, splicing, transport, and translation [14]. Unlike DNA methylation, RNA m^5C predominantly occurs in mRNA, non-coding RNAs, and small RNAs [15]. High-throughput sequencing (HTS) technologies, such as ultrafast bisulfite sequencing(UBS-seq) [16] and RNA-Bis-seq [17], are used to detect and analyze RNA m^5C by enabling the parallel sequencing of millions to billions of nucleotides, thus advancing research in genomics, transcriptomics, and epigenetics.

In this competition, our main task involves developing a workflow that systematically integrates a specialized bioinformatics pipeline for detecting RNA m^5C in HTS data. The core focus of this task is on the accuracy of m^5C site identification, false positive control, and computational efficiency.

Notably, we integrate several software components (e.g., hisat-3n, samtools, cutseq, and umicollapse) into a single docker. Starting from raw sequencing data (FASTQ files), the pipeline performs adapter trimming, low-quality sequence removal, rRNA/tRNA filtering, genome alignment, m^5C site detection, and stringent filtering. The final output is a high-confidence list of m^5C candidate sites with high accuracy and low false positives.

By analyzing the overall workflow, dissecting individual commands, and conducting flame graph analysis on the time-consuming steps, we have identified further opportunities for optimization. **Through adjusting parameters(Section 7.3.1), adopting file-based intermediary storage(Section 7.3.2), parallelizing the workflow and applying parallelization at the hardware level(Section 7.3.3), we successfully met the accuracy requirements while significantly reducing the overall workflow runtime. Specifically, the runtime was shortened from 6.9 hours to 1.33 hours, achieving a speedup of 5.188 times.**

Meanwhile, through further analysis and reflection, we have identified additional opportunities for optimization, which will become a key focus of our future exploration. Specifically, we will explore the potential of optimizing the compiler and compilation process, binding CPU cores, and parallelizing key code segments to enhance the overall performance of the workflow.

7.2. **Related Work.**

Parallelization is a critical approach to improving performance efficiency. By decomposing computational tasks into multiple independent sub-tasks, it enables concurrent execution across multiple processing units or computing nodes. This drastically reduces computation time, especially for large-scale data processing or complex computational tasks. Parallelization enhances hardware resource utilization and accelerates computation-intensive tasks, making it a cornerstone in fields such as HPC, deep learning, and big data analytics. Below are some commonly used parallelization methods.

(1) **Task Parallelism**
Task parallelism involves dividing a large workflow into independent tasks that can be processed concurrently. This method is widely used in high-performance computing (HPC) to

maximize resource utilization across multiple processors. For example, Amdahl [18] proposed a model for understanding the limitations of parallel computing, while Chiu [19]proposed a novel task-parallel pipeline programming framework called *Pipeflow*. By decoupling data abstractions from task scheduling, *Pipeflow* enables a more efficient implementation of task-parallel pipeline algorithms compared to existing frameworks. .

(2) **Data Parallelism**
Data parallelism focuses on splitting large datasets into smaller chunks and processing each part independently. This is particularly useful in applications like genomics, simulations, and image processing. For instance, Patterson and Hennessy [20] systematically evaluated the performance of data parallelism across different computational architectures using quantitative analysis methods. Their work provides crucial insights into task partitioning in genomics and high-performance computing. Culler and Singh [21] introduced parallel computer architecture principles that enhance data parallelism.

(3) **Multi-threading**
Multi-threading allows multiple threads to run concurrently within a single process. This is effective for tasks that can be divided into smaller, concurrent operations. For example, Tullsen et al. [22] explored simultaneous multithreading, a technique that allows multiple independent threads to issue instructions to multiple functional units in a superscalar processor during a single cycle. Lo et al. [23] proposed Simultaneous Multithreading (SMT), a technique that allows multiple threads to share all processor resources during each cycle, enabling the alternation between thread-level parallelism and instruction-level parallelism.

(4) **Vectorization**
Vectorization allows multiple data points to be processed simultaneously through hardware capabilities such as Single Instruction, Multiple Data(SIMD) [24]. This method is commonly used in high-performance computing for numerical simulations. For example, Markus et al. present pipeline vectorization for synthesizing hardware pipelines based on software vectorizing compilers and improved efficiency and ease of development of hardware designs [25].

7.3. **Proposed Methods.**

Our optimization goal is to minimize runtime while ensuring the accuracy and reliability of m^5C site detection. **The focus of the optimization is on tuning instruction parameters and implementing various parallelization strategies to enhance overall performance.** By employing various optimization methods, we achieved high parallelism across the entire workflow and built a new workflow on top of the original one. **Figure 36 provides a clear illustration of the overall framework of the new workflow. The key optimization of the novel workflow lies in parallelizing the `Site Calling` and `Filtering` steps, which allows the four TSV files containing statistics for each condition to be generated concurrently, significantly improving data generation efficiency. Additionally, we have parallelized `Stage 1` for the three datasets, fully leveraging the computational resources of multi-core processors and greatly reducing processing time. Furthermore, we enhance the overall process by enabling hardware parallelism through hyper-threading for deeper optimization.** These optimizations have resulted in a substantial performance improvement across the entire workflow, accelerating processing speed and greatly increasing data throughput. The specific details of each optimization method are as follows.

7.3.1. *Parameter Adjustment and Analysis.*

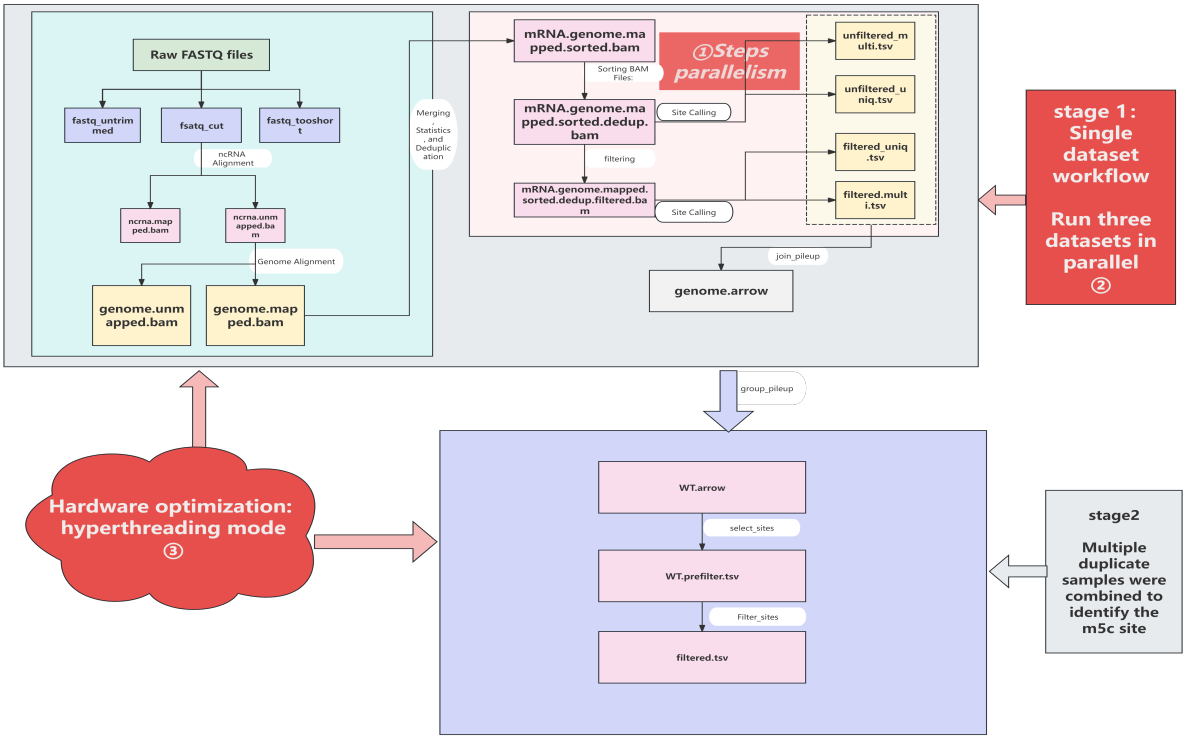


Figure 36. *RNA m^5C Optimized workflow*

In order to optimize data processing efficiency, improve detection accuracy, and enhance the relevance of the results, we adjusted and analyzed several key parameters in the commands throughout the workflow. The following is a detailed analysis and adjustment of the parameters. Each experiment focuses on tuning parameters for a single step from the SRR23538290 dataset. The optimal parameters chosen after parallelization may differ from those selected in the following results.

(1) Adjustment of Cutseq Tool Parameters:

- `-t 28`: Through testing different hread count configurations (illustrated in Figure 37), We chose to use **28** threads for data processing. Increasing the number of threads significantly speeds up data processing, especially when handling large-scale data, thus reducing overall runtime. The optimal data configuration results (**1m43s**) in a speedup of **1.58 times** compared to the least efficient data configuration result (**2m43s**).
- `-A INLINE`: The inline barcode mode was enabled to ensure barcode accuracy, which is crucial in multiplexed sample data, preventing barcode interference and reducing error rates.
- `-m 20`: A minimum sequence length of 20 was set to filter out short sequences, ensuring that the data used meets a certain quality standard, and avoiding the negative impact of short sequences on downstream analysis.
- `--trim-polyA`: PolyA tails were trimmed, a standard step in RNA-seq data processing that effectively removes interference from PolyA tails during alignment.
- `--ensure-inline-barcode`: This ensures the accuracy of inline barcodes, which is essential for multiplexed sample processing in high-throughput sequencing, avoiding confusion caused by barcode errors.

(2) Adjustment of Hisat-3n Tool Parameters:

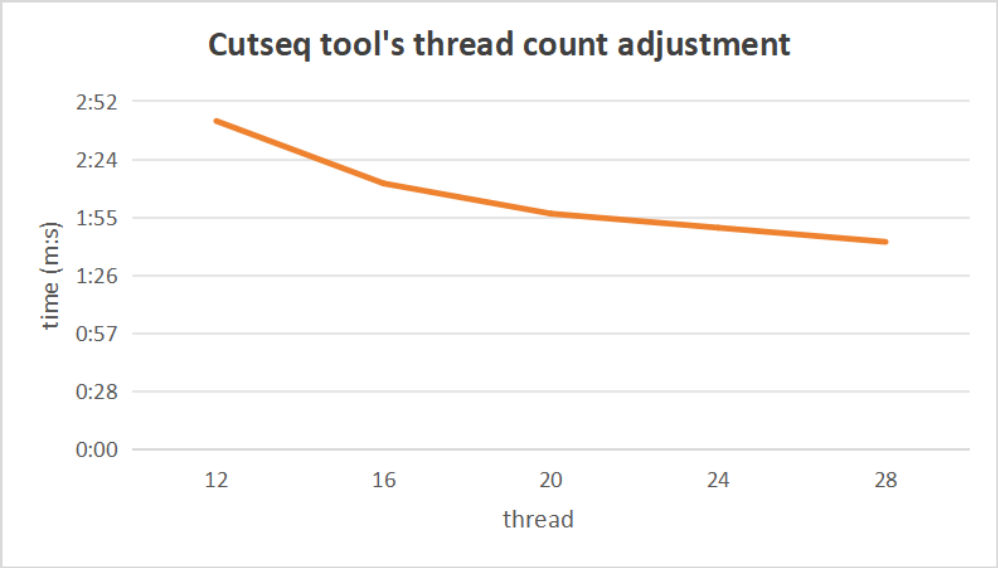


Figure 37. *Cutseq tool's thread count adjustment*

- `-p 16`: Based on experiments with various process settings (as shown in Figure 38), we used **16** threads for alignment, which improved alignment speed while ensuring sufficient computational resources for handling the data. The optimal data result (**4m47s**) achieves an **1.33-fold** speedup compared to the worst-performing data result (**6m23s**).
- `--base-change C,T`: This parameter converts cytosine (C) to thymine (T), which is crucial for the detection of RNA m^5C modifications.
- `--mp 8,2`: Mismatch and indel penalties were set to 8 and 2, respectively, allowing some degree of mismatch and structural variation, which improves alignment tolerance, especially in highly variable regions.
- `--no-spliced-alignment`: Spliced alignment was disabled, which is beneficial for small RNA data that does not require splice site information.
- `--directional-mapping`: Directional mapping was enabled to correctly process paired-end sequencing data.

(3) Adjustment of Samtools Tool Parameters:

- `-@ 28`: Through systematic experimentation with different process configurations (see Figure 39 for the detailed results), We specified **28 threads** for processing, improving data processing efficiency. The optimal data result (**21.193s**) is **1.34 times** faster compared to the worst data result (**28.427s**).
- `-e '!flag.unmap'`: This filters out unmapped reads, ensuring that only successfully aligned data are analyzed, reducing the interference from irrelevant data.
- `-O BAM`: BAM format was used for output, which is the standard format for genomic data processing, facilitating subsequent analysis and visualization.

(4) Adjustment of UMICollapse Tool Parameters:

- `-Xms8G -Xmx40G -Xss100M`: These JVM memory settings allocate up to 40GB of maximum heap memory. This helps prevent memory issues when processing large-scale data, ensuring stability during data processing.
- `-t 2`: We set 2 threads for the UMI deduplication process.

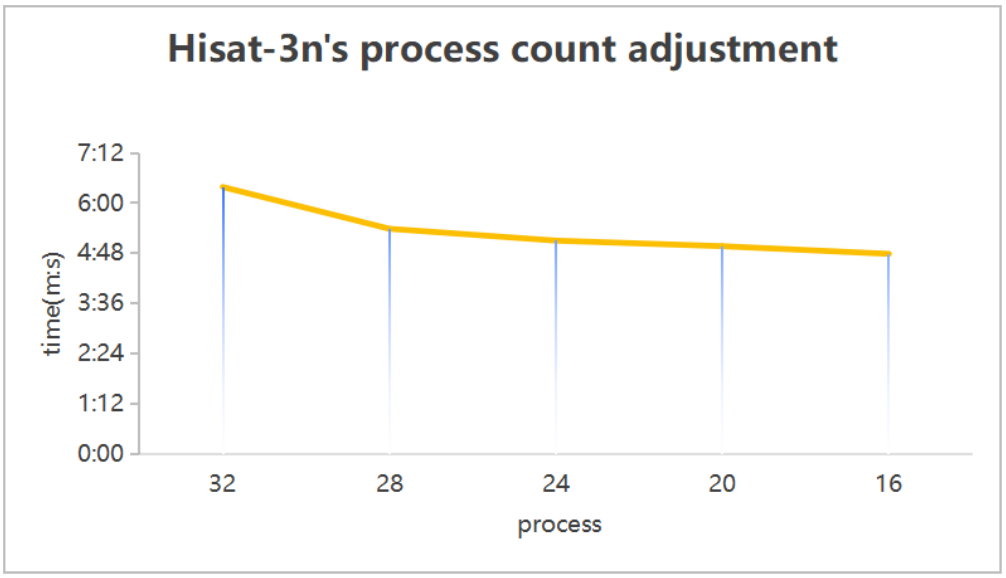


Figure 38. *Hisat-3n tool's process count adjustment*

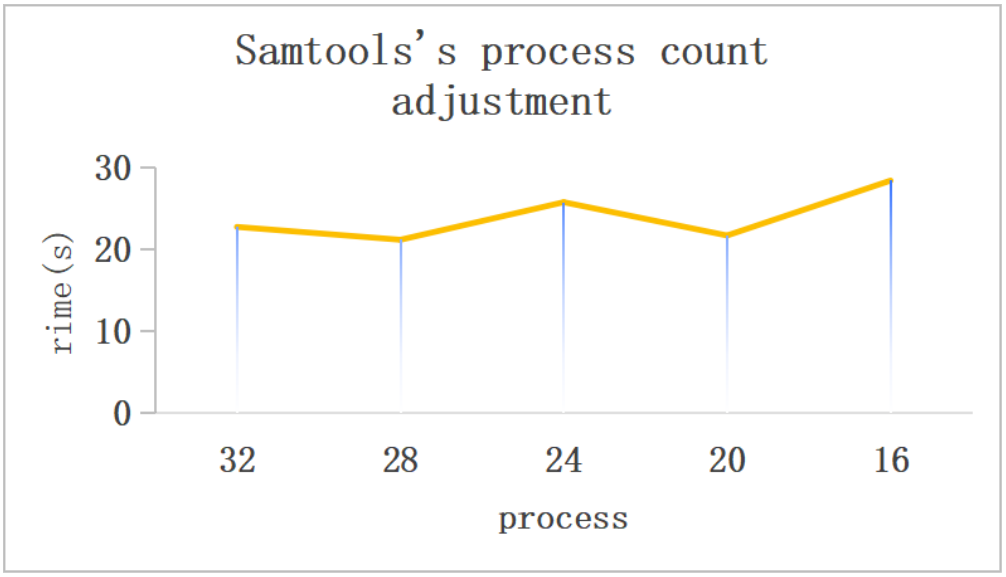


Figure 39. *Samtools tool's process count adjustment*

- `-T 16`: We used 16 threads for deduplication to accelerate the process, addressing the need for large-scale data handling.
- `--merge avgqual`: UMI sequences were merged using the average quality value, which improves deduplication accuracy.
- `--two-pass`: A two-pass deduplication strategy was enabled to enhance the reliability of the deduplication results.

After adjusting the parameters, we ensured the accuracy and relevance of the final results. Our experiments demonstrated that the optimal number of threads or processes for each step must be selected with caution. A setting that is too high can lead to excessive context switching or resource

contention, resulting in performance degradation. Conversely, a setting that is too low may not fully leverage the available computational resources, impacting overall efficiency. **Proper parameter selection can enhance system throughput, shorten execution time, and significantly improve overall performance.**

7.3.2. *Adopting file-based intermediary storage.*

This optimization strategy will focus on the following command(Here, we refer to it as the Site calling and filtering step.), using the SRR23538290 dataset as an example.

The original command is :

```
/code/samtools/samtools view -@ 16 -e "rlen<100000" -h
↪ /asc25/SRR23538290/SRR23538290.mRNA.genome.mapped.sorted.dedup.bam
↪ |/code/hisat-3n/hisat-3n-table -p 16 -u - --alignments
↪ --ref/asc25/ref/Homo_sapiens.GRCh38.dna.primary_assembly.fa  --output-name
↪ /dev/stdout --base-change C,T |cut -f 1,2,3,5,7 |gzip -c
↪ >/asc25/SRR23538290/SRR23538290_unfiltered_uniq.tsv.gz
```

The optimized command is:

```
/code/samtools/samtools view -@ 16 -e "rlen<100000" -h
↪ /asc25/SRR23538290/SRR23538290.mRNA.genome.mapped.sorted.dedup.bam >
↪ /asc25/SRR23538290/SRR23538290.unfiltered.bam
/code/hisat-3n/hisat-3n-table -p 16 -u --alignments
↪ /asc25/SRR23538290/SRR23538290.unfiltered.bam --ref
↪ /asc25/ref/Homo_sapiens.GRCh38.dna.primary_assembly.fa  --output-name
↪ /asc25/SRR23538290/SRR23538290.unfiltered.uniq.tsv --base-change C,T
cut -f 1,2,3,5,7 /asc25/SRR23538290/SRR23538290.unfiltered.uniq.tsv >
↪ /asc25/SRR23538290/SRR23538290.unfiltered_uniq.tsv
 gzip -c /asc25/SRR23538290/SRR23538290.unfiltered_uniq.tsv >
 ↪ /asc25/SRR23538291/SRR23538291_unfiltered_uniq.tsv.gz
```

In the original command, RNA/tRNA site calling and filtering are performed, and the data is passed through a pipeline (|) from `samtools` to `hisat_3n_table`, then to `cut`, and finally to `gzip`. The system's pipe and standard input/output streams may not be well optimized in terms of I/O buffering, leading to frequent synchronization and data transfer delays. **Additionally, the PCIe 4.0 consumer-grade high-end solid-state drive (the Hynix P41 1TB SSD) provides exceptionally high sequential read/write speeds, around $\sim$ 7 GB/s, significantly accelerating the storage and retrieval of intermediate results**. Compared to traditional hard drives, solid-state drives have distinct advantages in read/write speed, random access, and parallel processing. Therefore, we optimized the original process by storing intermediate results and using them as input for the next step, which fully leverages the performance of the hard drive and avoids synchronization and latency issues inherent in pipe transmission.

The performance comparison results after optimization are shown in Table 16. The **original version** refers to the execution time of the individual step when running the `hisat-3n-table` command on each dataset sequentially, while the **optimized version** refers to the execution time after implementing the *Adopting file-based intermediary storage* strategy based on the original

version. In addition, the data in parentheses represents the **speedup ratio**. According to the data in Table 16, **this approach yields a speed enhancement ranging from 1.76 times to 2.00 times, with an average of 1.91 times, across the three given datasets.**

Adopting file-based intermediary storage shortens runtime for the following reasons:

(1) **Enhanced I/O Performance:** Pipelines introduce I/O overhead, especially with large datasets. By storing intermediate results in files, disk I/O is optimized, reducing data transfer delays between processes, which is more efficient for large data.
(2) **Eliminated redundant data flow:** Data passing through multiple pipeline commands incurs extra I/O costs. Storing intermediate results in files avoids redundant processing and reduces data movement between memory and CPU.
(3) **Lower memory usage:** Pipeline processing requires storing and transferring data in memory, which can cause bottlenecks with large datasets. Using files for intermediate results reduces memory load, mitigating performance issues.

Table 16. *The Performance comparison results for a single step of "site calling and filtering" after adopting the file-based intermediary storage strategy*

Dataset / Version	**SRR23538290**	**SRR23538291**	**SRR23538292**	**Total time of calling and filtering step**
Original	1h 27m 8s	1h 49m 49s	1h 42m 42s	4h 59m 39s
Optimized	**0h 49m 21s**(**1.76**×)	**0h 54m 53s**(**2.00**×)	**0h 52m 21s**(**1.96**×)	**2h 36m 35s** (**1.91**×)

7.3.3. *Process Parallelism.*

Process parallelism refers to the concurrent execution of distinct and independent subtasks, each with different functions, in order to enhance the overall processing efficiency of the computing system and significantly reduce the total execution time. Especially in large-scale computational tasks, process parallelism can effectively distribute the computational load, alleviating the time bottleneck caused by serial execution, thereby improving both the execution speed and resource utilization of the task.

(1) **Steps parallelism:** In the processing pipeline of a single dataset, multiple independent processing steps were identified. We have attempted to parallelize these steps in order to enhance processing efficiency and reduce overall computation time, **as the Optimization of Part 1 (steps parallelism) shown in Figure 36**. By parallelizing the tasks, we can fully exploit computational resources, mitigate performance bottlenecks caused by serial execution, and thus accelerate the data processing workflow, optimizing the overall system performance.

In the script, we accelerate the task execution by launching multiple processes in parallel using the & operator, as detailed below:

(a) **Parallel execution of four `hisat-3n-table` processes**: Each process handles a different BAM file and generates the corresponding TSV output file.
(b) **Parallel execution of four `cut` processes**: Each process extracts specific columns from the TSV files and generates new files.
(c) **Parallel execution of four `gzip` processes**: Each process compresses the TSV files that have been cut.

By parallelizing these tasks, we significantly reduce the execution time for each stage, thereby improving the overall data processing efficiency. The `wait` command ensures that

Table 17. *The performance comparison results of multi steps of "site calling and filtering" after optimizing for Steps parallelism.*

Version \ Dataset	SRR23538290	SRR23538291	SRR23538292	Total time of site calling and filtering step
Original	0h 49m 21s	0h 54m 53s	0h 52m 21s	2h 36m 35s
Optimized	**0h 14m 53s(3.31×)**	**0h 17m 53s(3.06×)**	**0h 16m 39s(3.14×)**	**0h 49m 25s (3.16×)**

each set of parallel processes completes before moving to the subsequent tasks. The performance comparison results after optimization are shown in Table 17. The **original version** refers to the total execution time of sequentially running four `hisat-3n-table` commands on each dataset, while the **optimized version** refers to the execution time after implementing steps parallelism to run the `hisat-3n-table` commands in parallel based on the original version. In addition, the data in parentheses represents the **speedup ratio**. According to the data in Table 17, **this approach yields a speed enhancement ranging from 3.06 times to 3.31 times, with an average of 3.17 times, across the three given datasets.**

(2) **Datasets parallelism:** Since we need to process three independent datasets at stage1, it is apparent that we can parallelize the processing of these datasets within the script, which would significantly reduce the overall runtime.

After applying dataset parallelism, **corresponding to the Optimization of part 2 (dataset parallelism) shown in Figure 36**, the runtime is significantly reduced. The performance comparison results after optimization are shown in Table 18. **Original version** refers to the execution time of the stage1 step for the three datasets when executed sequentially based on the aforementioned optimization, while the **optimized version** refers to the execution time after implementing parallel processing for the three datasets based on the original version. In addition, the data in parentheses represents the **speedup ratio**. Due to factors such as resource contention, thread scheduling overhead, and imbalanced task allocation, the execution time for a single dataset in parallel processing may be slightly longer compared to serial execution. However, in parallel execution, the total runtime is determined by the dataset with the longest processing time, while in serial execution, the total runtime is the sum of the individual processing times of all datasets. **Consequently, despite the slightly increased runtime for each individual dataset, parallel execution results in a significantly shorter total runtime compared to serial execution, yielding an overall speedup of approximately 2 times**.

(3) **Hardware parallelism:**

Hyper-Threading is a hardware parallelism technology introduced by Intel, which allows each physical processor core to simulate multiple logical cores, thereby improving the efficiency of multi-threaded tasks. By virtualizing each physical core into two logical cores, Hyper-Threading can execute more threads concurrently, enhancing the utilization of CPU resources and significantly boosting system IO performance, especially when handling computations and file IO asynchronously.

The m5c site detection process involves extensive database operations, data analysis, and scientific computations. Enabling hyper-threading is highly beneficial for such multi-threaded tasks, **as the Optimization of Part 3 (Hardware parallelism) shown in Figure 36**, with the following specific advantages:

(a) **Improved Resource Utilization**: Hyper-Threading simulates multiple threads, enabling better utilization of processor resources, such as computation units and cache, thus increasing core throughput.
(b) **Enhanced Multi-threaded Performance**: In multi-threaded applications, Hyper-Threading enables each physical core to handle tasks like multiple logical cores, thus improving parallel execution efficiency.
(c) **Reduced Response Time**: In multi-tasking systems, Hyper-Threading helps reduce task queuing times, enhancing system responsiveness.

The experiment shows that, based on the aforementioned optimization, enabling hyper-threading reduces the total execution time of the workflow from **1.83 hours** to **1.33 hours**, achieving a speedup of approximately **1.376 times**.

Table 18. *The performance comparison results of stage 1 after optimizing for datasets parallelism based on step parallelism..*

Dataset / Version	**SRR23538290**	**SRR23538291**	**SRR23538292**	**Total time of stage1**
Sequential Execution	0h 47m 26s	0h 59m 16s	0h 53m 22s	2h 41m 55s
with Dataset Parallelism	**1h 5m 31s(0.72×)**	**1h 19m 9s(0.74×)**	**1h 13m 42s(0.72×)**	**1h 20m 49s(2.00×)**

7.4. **Experiments.**

7.4.1. *Setup.*

Hardware

Our experimental results are based on a Linux server, our CPU is AMD EPYC 9124 16-Core Processor@3.7GHZ; our GPU is two NVIDIA GeForce RTX 4070 Ti Super, our hard drive model is Hynix P41 1TB SSD, with sequential read and write speeds of approximately 7 GB/s, with a total memory size of 128GB.

Software

Our server software environment is the operating system is Ubuntu 22.04.4 LTS version, Python 3.10.12, CUDA v12.4.

Datasets

This study utilizes the dataset provided by the official competition organizers, which includes three raw sequencing data samples (FASTQ files) of wild-type HeLa polyA+ RNA treated with UBS.

7.4.2. *Accuracy Evaluation.*

Precision Evaluation

Precision of m^5C site is defined as the proportion of correctly detected sites (true positives, TP) among all detected sites. It is calculated as:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Where TP represents the sites that match the standard reference, and FP represents the sites that do not match the standard.

After validation and evaluation of the results from the filtered TSV files of three datasets (e.g., SRR23538290.filtered.tsv, SRR23538291.filtered.tsv, and SRR23538292.filtered.tsv) by the organizing committee, the final **Precision** of the m^5C sites was **95.44%**, meeting specified thresholds of precision (**>=95%**) with the standard reference set of m5C sites.

Correlation Evaluation

Correlation is computed by calculating the Pearson correlation coefficient of the unconverted ratios (ur) for the overlapping sites between the participant's detected sites and the true sites. The formula is as follows:

$$\mathrm{r} = \frac{\sum_{k=1}^{n} \left(ur_{\text{true},k} - \overline{ur_{\text{true}}}\right)\left(ur_{\text{detected},k} - \overline{ur_{\text{detected}}}\right)}{\sqrt{\sum_{k=1}^{n} \left(ur_{\text{true},k} - \overline{ur_{\text{true}}}\right)^2}\sqrt{\sum_{k=1}^{n} \left(ur_{\text{detected},k} - \overline{ur_{\text{detected}}}\right)^2}}$$

Similarly, after the evaluation of the filtered TSV files by the organizing committee, the final **Correlation coefficient** of the m^5C sites was **99.86%**, meeting specified thresholds of correlation (**>=90%**) with the standard reference set of m5C sites.

Consequently, our precision and correlation values are 95.44% and 99.86% respectively, which satisfy the criteria of a precision exceeding 95% and a correlation surpassing 90%.

7.4.3. *Performance Evaluation.*

The runtime of the entire workflow is a key metric for evaluating its performance. After multiple rounds of performance optimization, we achieved the best performance results to date. **The running time of each step in the original workflow and the optimized workflow are presented in Table 19 and Table 20, respectively. In these tables, the original workflow takes a total of 6h 54m 6s to execute, while the optimized workflow takes a total of 1h 19m 9s to execute.**

Table 19. *Performance results without optimizing for steps parallelism and datasets parallelism. Total running time is* ***6h 54m 6s(***$1h59m15s+2h33m32s+2h19m28s=6h54m6s$***)***

Step \ Dataset	**SRR23538290**	**SRR23538291**	**SRR23538292**
cutseq	2m 3s	2m 57s	2m 34s
hisat-3n-index	4m 54s	6m 38s	5m 57s
samtools-fsatq	0m 32s	0m 32s	0m 25s
hisat-3n-index	8m 4s	10m 23s	9m 15s
samtools-sort	1m 58s	2m 57s	2m 22s
samtools-view	0m 9s	0m 14s	0m 12s
java-server	13m 49s	19m 8s	17m 21s
samtools-index	0m 12s	0m 13s	0m 11s
hisat-3n-table	22m 8s	27m 2s	25m 42s
hisat-3n-table	20m 49s	26m 44s	24m 52s
samtools-view	1m 1s	2m 43s	2m 40s
hisat-3n-table	22m 37s	27m 3s	25m 34s
hisat-3n-table	20m 33s	26m 17s	23m 54s
join_pileup.py	0m 36s	0m 41s	0m 39s
complete flow	1h 59m 15s	2h 33m 32s	2h 19m 28s

Table 20. *The performance results after optimizing for steps parallelism and datasets parallelism. The max runtime value of the "Complete flow" means the total runtime(**1h 19m 9s**)*

Step \ Dataset	**SRR23538290**	**SRR23538291**	**SRR23538292**
cutseq	5m1s	6m49s	6m4s
hisat-3n –index	8m30s	10m54s	9m59s
samtools	0m36s	0m57s	0m44s
hisat-3n –index	16m44.302s	19m1.839s	18m17.075s
samtools sort	3m31.296s	2m11.857s	2m18.497s
samtools view	0m20.264s	0m8.121s	0m9.747s
java UMICollapse	12m31.675s	17m22.680s	15m11.706s
samtools index	0m5.674s	0m20.039s	0m13.642s
samtools view	0m29.343s	0m49.021s	0m30.209s
samtools view	0m59.640s	2m56.766s	2m0.479s
samtools view	0m28.939s	0m49.988s	0m33.705s
hisat-3n-table	14m48.447s	15m37.510s	16m14.753s
cut	0m4.432s	0m3.463s	0m4.785s
gzip	0m24.032s	0m21.128s	0m24.009s
join_pileup	0m52.432s	0m41.148s	0m51.932s
Complete flow	1h 5m 31s	**1h 19m 9s**	1h 13m 42s

7.5. **Optimization Plans.**

(1) Change the compile and compile options

Optimizing compiler options and processes is a crucial approach to enhancing program performance and efficiency. By properly configuring compiler optimization flags, adjusting linking options, parallelizing the compilation process, and performing hardware-specific optimizations, we can significantly improve execution speed, memory utilization, and computational capacity. We have already tried using different compilers (such as the AMD compiler that matches the server) and various compiler optimization flags. However, these optimizations did not yield the expected results. Therefore, we plan to explore additional optimization strategies. Below are some potential avenues for improvement:

(a) **Compiler Optimization Flags** Compiler optimization flags allow various optimizations during the compilation process to improve program performance. These flags control how the code is generated, and can reduce memory consumption and improve execution efficiency. In addition to the commonly used flags such as `-O2`, `-O3`, and `-funroll-loops`, the following options can also be considered:

- `-march=native`: Optimizes the code based on the hardware features of the current machine, allowing the program to fully leverage the CPU instruction set (e.g., AVX, SSE).
- `-ftree-vectorize`: Automatically vectorizes the code, utilizing SIMD (Single Instruction, Multiple Data) instructions to accelerate computations on supported processors.

(b) **Link-time Optimization (LTO)** Link-time optimization (LTO) performs cross-file optimizations during the linking phase, allowing the compiler to optimize across modules, further improving performance.

- `-flto`: This option enables LTO, optimizing unused code, eliminating redundant functions, and enhancing overall performance through cross-module optimization.

(c) **Profile-Guided Optimization (PGO)** Profile-guided optimization (PGO) optimizes the most frequently used parts of the program by collecting runtime performance data. It typically involves two steps:
- First, compile the program with `-fprofile-generate` to gather performance data.
- Second, recompile the program with `-fprofile-use` based on the collected data to optimize performance bottlenecks.

(d) **Optimized Math and Threading Libraries** For compute-intensive applications, using optimized math libraries and threading libraries can significantly boost program performance:
- Use optimized math libraries such as `Intel MKL` or `OpenBLAS` to accelerate matrix operations and other numerical computations.
- Use threading libraries like `OpenMP` or `Intel Threading Building Blocks (TBB)` for efficient parallel computing support.

(2) Binding CPU cores

When multiple tasks are executed in parallel, occupying the entire CPU, binding CPU cores (i.e., assigning specific processes or threads to fixed CPU cores) offers several advantages:

(a) **Reduced Cache Misses**: By binding tasks to specific cores, data can remain resident in the cache of that core, which reduces the frequency of cache misses and enhances data access speed.

(b) **Minimized Context-Switching Overhead**: Binding tasks to designated cores reduces the frequency of task migrations between cores, thus minimizing the performance penalties associated with context switching.

(c) **Reduced Resource Contention**: By allocating tasks to different cores, competition for shared resources (such as L3 cache and memory bandwidth) between cores is minimized, improving overall system performance.

(d) **Improved Real-Time Performance**: Binding critical tasks to specific cores ensures they are not disturbed by other tasks, thus fulfilling real-time performance requirements.

In conclusion, this strategy may potentially enhance the execution efficiency of certain tasks.

(3) Parallel processing in $hisat_3n_table.cpp$

We performed a flame graph analysis on one of the time-consuming steps, the `hisat-3n-table` step, as shown in Figure 40, and identified that the performance bottleneck primarily lies in the locking mechanisms within the producer-consumer model. To address this issue, we plan to optimize the producer part of the code. Specifically, we will conduct preliminary processing and filtering of each line in the input file and place the processed data into a thread pool for further analysis by the consumer. To accelerate the producer's processing speed, we intend to process each line of data in the input file in parallel based on chromosomes, thereby reducing competition and wait times for the locking mechanisms, which

will increase the throughput of the producer and optimize overall performance. Theoretically, by implementing these optimizations, we expect to improve the program's execution efficiency and shorten its runtime.

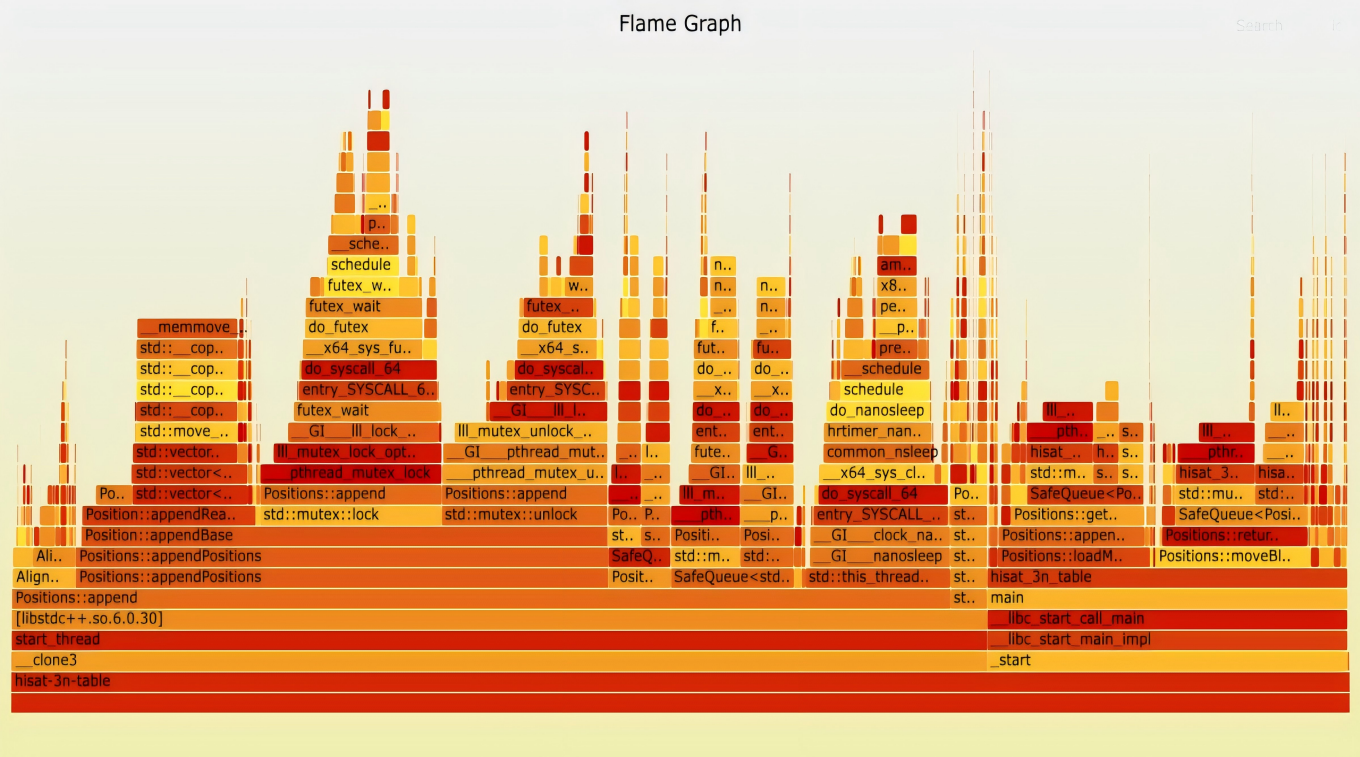


Figure 40. *Flame graph analysis of the hisat-3n-table step*

7.6. **Conclusion.**

In this proposal, we employed optimization methods such as parameter tuning, process optimization, and process parallelization. As a result, the execution time for the three given datasets was reduced from 6.9 hours to 1.33 hours, achieving a 5.188 times improvement. Similarly, we analyzed the overall process and identified the critical bottleneck process as being related to the execution of the `Hisat_3n_table.cpp` program. Due to time constraints, the parallelization optimization of the `Hisat_3n_table.cpp` program still needs to be improved. Additionally, after analysis, optimizations such as compilation and thread parallelization will be implemented.

Acknowledge

First of all, we would like to express our gratitude to the organizing committee for giving us the opportunity to participate in this competition. Secondly, we are grateful to the Computer College of Xiangtan University for providing a comfortable environment during the competition. We are also thankful to the High Performance Computing Platform of Xiangtan University. We are grateful to our advisor, Peiying Lin, for her strong support throughout the competition. We are also grateful to the other teachers who answered our questions and provided us with great help.

References

[1] X. Xu, X. Yue, R. Mao, Y. Deng, S. Huang, H. Zou, X. Liu, S. Hu, C. Feng, S. Shu *et al.*, "Jxpamg: a parallel algebraic multigrid solver for extreme-scale numerical simulations," *CCF Transactions on High Performance Computing*, vol. 5, no. 1, pp. 72–83, 2023.

[2] Z. Zhao, "A numerical simulation of chemical flooding reservoirs for heterogeneous systems," Master's thesis, Xiangtan University, 2023.

[3] W. Guan, C. Qiao, H. Zhang, C.-S. Zhang, M. Zhi, Z. Zhu, Z. Zheng, W. Ye, Y. Zhang, X. Hu *et al.*, "On robust and efficient parallel reservoir simulation on tianhe-2," in *SPE Reservoir Characterisation and Simulation Conference and Exhibition?* SPE, 2015, p. D031S017R004.

[4] J. Zhou, X. Hu, L. Zhong, S. Shu, and L. Chen, "Two-grid methods for maxwell eigenvalue problems," *SIAM journal on numerical analysis*, vol. 52, no. 4, pp. 2027–2047, 2014.

[5] J. Abramson, J. Adler, J. Dunger, R. Evans, T. Green, A. Pritzel, O. Ronneberger, L. Willmore, A. J. Ballard, J. Bambrick *et al.*, "Accurate structure prediction of biomolecular interactions with alphafold 3," *Nature*, pp. 1–3, 2024.

[6] S. Cheng, X. Zhao, G. Lu, J. Fang, Z. Yu, T. Zheng, R. Wu, X. Zhang, J. Peng, and Y. You, "Fastfold: Reducing alphafold training time from 11 days to 67 hours," *arXiv preprint arXiv:2203.00854*, 2022.

[7] J. Su, C. Han, Y. Zhou, J. Shan, X. Zhou, and F. Yuan, "Saprot: Protein language modeling with structure-aware vocabulary," *bioRxiv*, pp. 2023–10, 2023.

[8] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro *et al.*, "Efficient large-scale language model training on gpu clusters using megatron-lm," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–15.

[9] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–16.

[10] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.

[11] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.

[12] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," *arXiv preprint arXiv:2011.13456*, 2020.

[13] G. Hinton, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[14] Y. Motorin, F. Lyko, and M. Helm, "5-methylcytosine in rna: detection, enzymatic formation and biological functions," *Nucleic Acids Research*, vol. 38, no. 5, pp. 1415–1430, 12 2009. [Online]. Available: https://doi.org/10.1093/nar/gkp1117

[15] Q. Zhang, F. Liu, W. Chen, H. Miao, H. Liang, Z. Liao, Z. Zhang, and B. Zhang, "The role of rna m5c modification in cancer metastasis," *International journal of biological sciences*, vol. 17, no. 13, p. 3369, 2021.

[16] Q. Dai, C. Ye, I. Irkliyenko, Y. Wang, H.-L. Sun, Y. Gao, Y. Liu, A. Beadell, J. Perea, A. Goel *et al.*, "Ultrafast bisulfite sequencing detection of 5-methylcytosine in dna and rna," *Nature Biotechnology*, pp. 1–12, 2024.

[17] Y. Hu, C. Chen, X. Tong, S. Chen, X. Hu, B. Pan, X. Sun, Z. Chen, X. Shi, Y. Hu *et al.*, "Nsun2 modified by sumo-2/3 promotes gastric cancer progression and regulates mrna m5c methylation," *Cell death & disease*, vol. 12, no. 9, p. 842, 2021.

[18] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities, reprinted from the afips conference proceedings, vol. 30 (atlantic city, n.j., apr. 18–20), afips press, reston, va., 1967, pp. 483–485, when dr. amdahl was at international business machines corporation, sunnyvale, california," *IEEE Solid-State Circuits Society Newsletter*, vol. 12, no. 3, pp. 19–20, 2007.

[19] C.-H. Chiu, Z. Xiong, Z. Guo, T.-W. Huang, and Y. Lin, "An efficient task-parallel pipeline programming framework," in *Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region*, ser. HPCAsia '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 95–106. [Online]. Available: https://doi.org/10.1145/3635035.3635037

[20] J. L. Hennessy and D. A. Patterson, *Computer Architecture, Fifth Edition: A Quantitative Approach*, 5th ed. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2011.

[21] D. Culler, J. P. Singh, and A. Gupta, *Parallel Computer Architecture: A Hardware/Software Approach*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1998.

[22] D. M. Tullsen, S. J. Eggers, and H. M. Levy, "Simultaneous multithreading: maximizing on-chip parallelism," in *Proceedings of the 22nd Annual International Symposium on Computer Architecture*, ser. ISCA '95. New York, NY, USA: Association for Computing Machinery, 1995, p. 392–403. [Online]. Available: https://doi.org/10.1145/223982.224449

[23] J. L. Lo, J. S. Emer, H. M. Levy, R. L. Stamm, D. M. Tullsen, and S. J. Eggers, "Converting thread-level parallelism to instruction-level parallelism via simultaneous multithreading," *ACM Trans. Comput. Syst.*, vol. 15, no. 3, p. 322–354, Aug. 1997. [Online]. Available: https://doi.org/10.1145/263326.263382

[24] A. E. Eichenberger, P. Wu, and K. O'brien, "Vectorization for simd architectures with alignment constraints," *Acm sigplan notices*, vol. 39, no. 6, pp. 82–93, 2004.

[25] M. Weinhardt and W. Luk, "Pipeline vectorization," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 20, no. 2, pp. 234–248, 2001.